Parallel Numerical Simulation of Electromagnetic Wave Propagation Using the Finite-Difference Time-Domain Method

Thesis v1.0 (Translated)

Ivan Janevski

September 2026

Contents

1 Introduction **4**
- 1.1 Motivation and Scope . . . 4
- 1.2 The Problem of Numerical Electromagnetic Simulation . . . 4
- 1.3 The Finite-Difference Time-Domain Method . . . 5
- 1.4 The Need for Parallelization . . . 5
- 1.5 Thesis Organization . . . 5

2 Theory of Electromagnetic Wave Propagation **7**
- 2.1 Maxwell's Equations and Constitutive Relations . . . 7
- 2.2 One-Dimensional Model . . . 8
- 2.3 The E_z and H_y Field Components . . . 8
- 2.4 System of Differential Equations . . . 9
- 2.5 Symmetric System of Equations . . . 9
- 2.6 Material Properties . . . 9

3 Discretization of Maxwell's Equations **11**
- 3.1 The Finite-Difference Method and the Yee Grid . . . 11
- 3.2 Spatial and Temporal Discretization . . . 12
- 3.3 Discrete System of Equations . . . 13
- 3.4 The E_z and H_y Update Equations . . . 13
- 3.5 Update Coefficients and Material Modeling . . . 14
- 3.6 Excitation Pulse and Source Injection . . . 14
- 3.7 The Courant Condition and Time-Step Organization . . . 15

4 The Perfectly Matched Layer **16**
- 4.1 The Need for Absorbing Boundary Conditions . . . 16
- 4.2 The Basic Principle of the PML Method . . . 16
- 4.3 The Matching Condition . . . 17
- 4.4 PML Parameters and Coefficients . . . 17

5 Software Implementation **19**
- 5.1 Software Architecture . . . 19
- 5.2 The Static Library and Public API . . . 20
- 5.3 Simulation Representation and Data Structures . . . 20
- 5.4 FDTD Algorithm Implementation . . . 21
- 5.5 PML Integration . . . 22
- 5.6 Memory Management . . . 23
- 5.7 Simulation Configuration and Execution . . . 23
- 5.8 Visualization and Performance Measurement . . . 24

5.9 State Format and Simulation Resumption 24
5.10 Input Validation and Error Handling 24
5.11 Build Process and Distribution Structure 25

6 Parallelization with OpenMP **26**
6.1 Parallel Execution Model 26
6.2 Dependency Analysis and Work Distribution 26
6.3 Parallelization of the FDTD Algorithm 27
6.4 Synchronization and Thread Management 28
6.5 Memory Access Locality 29
6.6 Speedup Limitations 29
6.7 Arithmetic Intensity of the FDTD Algorithm 30
6.8 Choosing Between Serial and Parallel Execution 30

7 Experimental Methodology **31**
7.1 Hardware and Software Environment 31
7.2 Simulation Parameters 32
7.3 Performance Measurement Methodology 32
7.4 OpenMP Environment Control 33
7.5 Performance Metrics 33
7.6 Verification of Numerical Correctness 33
7.7 Statistical Processing of Repeated Measurements 34
7.8 Threats to Experimental Validity 34
7.9 Reproducing the Measurements 35

8 Results and Analysis **36**
8.1 Electromagnetic Wave Propagation 36
8.2 Analysis of PML Performance 37
8.3 Performance of the Serial Implementation 39
8.4 Speedup and Parallel Efficiency 39
8.5 Effect of Thread Count 40
8.6 Effect of Simulation Domain Size 41
8.7 Analysis of Performance Limitations 43

9 Conclusion **45**
9.1 Achieved Results 45
9.2 Contributions of the Work 45
9.3 Limitations of the Implemented Solution 46
9.4 Directions for Future Development 46
9.5 Recommendations for Use 46

References **48**

List of Abbreviations **49**

List of Figures **50**

List of Tables **51**

A Notation and Symbols **52**

B Derivation of the One-Dimensional Model **53**
B.1 The Nabla Operator . . . 53
B.2 Time-Domain Form of the Equations . . . 54
B.3 Expansion of the Curls and Scalar Form of the Equations . . . 54
B.4 One-Dimensional System of Equations . . . 55
B.5 Magnetic Currents and a Symmetric System of Equations . . . 55

C Pseudocode for the Finite-Difference Method Implementation **57**
C.1 Constants and Parameters . . . 57
C.2 Arrays . . . 57
C.3 Function Declarations . . . 58
C.4 Simulation Initialization . . . 58
C.5 Setting Fields and Materials . . . 59
C.6 Configuring the Boundary Layer . . . 60
C.7 Calculating Update Coefficients . . . 61
C.8 Updating Fields and Excitations . . . 62
C.9 Main Function . . . 63

D Project Architecture and Organization **65**
D.1 Project Directory . . . 65
D.2 Distribution Directory . . . 66
D.3 Results Directory . . . 67
D.4 Data Structures . . . 68

E Using the Software **70**
E.1 Using the Library . . . 70
E.2 Using the Program . . . 71
E.3 Using the Benchmark . . . 71

Chapter 1

Introduction

This work addresses the parallel numerical simulation of electromagnetic wave propagation using the finite-difference time-domain method. This chapter defines the simulation problem, while the following chapters gradually derive a specific numerical model, describe its implementation, and examine its parallelization.

1.1 Motivation and Scope

Reliable prediction of electromagnetic wave behavior is important in the development of antennas, transmission lines, sensors, and numerous electronic devices. Electromagnetic wave behavior is described by Maxwell's equations, formulated during the nineteenth century [1]. These equations can generally be solved either analytically or numerically.

Analytical solutions to Maxwell's equations are available only for a limited number of relatively simple problems with suitable symmetry. Once different materials, boundary conditions, or time-varying excitations are introduced, practical calculations become considerably more difficult. A numerical model is therefore required.

This work considers a one-dimensional model with the E_z and H_y components, which clearly demonstrates the most important properties of the method without the complexity of a three-dimensional grid. The model still supports the simulation of propagation, reflection, material losses, and absorption at the boundaries. It provides a suitable basis for understanding the algorithm and examining its parallel execution.

1.2 The Problem of Numerical Electromagnetic Simulation

An electromagnetic field varies continuously in space and time, whereas a computer can process only a finite number of values. In a numerical simulation, physical space is divided into cells and time into steps, so the numerical result inevitably represents an approximation. The main challenge is to ensure that this approximation remains stable and sufficiently faithful to the physical wave.

A finer grid represents the wave more accurately, but it also requires more memory and a greater number of operations. The time step cannot be increased freely to accelerate the computation because it is linked to the spatial step by a stability condition. A reliable simulation therefore requires a justified choice of parameters, rather than merely the largest possible number of samples.

A particular problem arises at the ends of a finite grid, where the wave may be artificially reflected and return to the observed region. Such a reflection can resemble a real physical phenomenon and lead to an incorrect interpretation of the results. For this reason, an absorbing layer that approximates the continuation of open space is introduced into the model.

1.3 The Finite-Difference Time-Domain Method

The finite-difference time-domain (FDTD) method provides a direct solution to the problem described above [2]. Derivatives in Maxwell's equations are replaced by differences between neighboring samples, transforming complex continuous dynamics into a sequence of simple updates. This formulation makes it possible to establish a clear connection between the physical model and the corresponding program expressions.

The electric and magnetic fields are placed at staggered points on the Yee grid [3]. At each step, the magnetic field is first calculated from the electric field, after which the updated magnetic state is used to update the electric field [4]. Bérenger's perfectly matched layer (PML) is used as the absorbing layer at the ends of the grid [5].

A particular advantage of the FDTD method is that it directly represents the evolution of a wave over time and does not require the solution of a large system of equations. In a single simulation, a short Gaussian pulse can excite a broad frequency range and reveal how different wave components propagate through the grid. The method is therefore well suited to physical analysis, visualization, and the development of efficient software.

1.4 The Need for Parallelization

The suitability of the FDTD method for simulation does not mean that its execution is always fast. Each time step processes almost every cell, so the total number of operations grows with both the grid length and the simulation duration. For fine or long-running models, computation time can limit the number of experiments that can be performed.

Updates to different cells within a single phase have separate destinations, which allows them to be distributed among multiple processor threads. This parallelism is introduced through the Open Multi-Processing (OpenMP) programming model and a small number of directives, while preserving a clear sequential structure [6]. In this way, multiple cores of a single computer are used for the same physical computation.

Parallelization does not, however, always produce a speedup proportional to the number of threads, because thread creation and synchronization introduce overhead. Amdahl's law further shows that the sequential part of a program imposes an upper bound on speedup regardless of the number of available processing units [7]. Consequently, overhead may outweigh the benefit for small grids, while shared memory may limit performance for large grids, as demonstrated by the analysis in this work.

1.5 Thesis Organization

This chapter has introduced the practical problem and explained the need for a numerical method and parallel execution. Chapter 2 begins with Maxwell's equations and

derives the one-dimensional system for E_z and H_y. Chapter 3 discretizes this system and obtains expressions that can be applied directly in the implementation.

Chapter 4 introduces the PML layer, which represents open space on a finite grid. Chapter 5 connects the mathematical model with the data structures, library, and sequential kernel implemented in the C programming language. Chapter 6 analyzes dependencies among iterations and presents the distribution of independent spatial iterations using the OpenMP model.

Chapter 7 defines the experimental environment, parameters, and reproducible measurement procedure. Chapter 8 analyzes visualizations of the fields and the measured performance indicators: execution time, throughput, speedup, and efficiency. Chapter 9 summarizes the results, identifies the limitations of the implementation, and proposes directions for further development.

Appendix A provides a table of the physical and numerical quantities used in the thesis. Appendix B presents a detailed derivation of the one-dimensional model, while Appendix C gives pseudocode for the finite-difference time-domain implementation. Appendix D describes the project architecture, data structures, and organization of the source code, distribution, and results, and Appendix E explains how to use the library, programs, and performance measurement tools.

Chapter 2

Theory of Electromagnetic Wave Propagation

The introduction explained why we use the FDTD method to simulate electromagnetic waves. In this chapter, we start from Maxwell's equations and derive a one-dimensional model for the E_z and H_y fields.

2.1 Maxwell's Equations and Constitutive Relations

Maxwell's equations relate the electric and magnetic fields to the charge and current in the medium under consideration [1]. Gauss's law describes the electric flux produced by electric charge, while the corresponding law for magnetism expresses the absence of magnetic monopoles. These two relations impose spatial constraints on the fields.

$$\nabla \cdot \vec{D} = \rho \tag{2.1}$$

$$\nabla \cdot \vec{B} = 0 \tag{2.2}$$

Faraday's law relates the change in magnetic flux density to the curl of the electric field. The Ampère-Maxwell law relates electric current and the change in electric flux density to the curl of the magnetic field. These two curl equations are subsequently used to implement the simulation.

$$\nabla \times \vec{E} = -\frac{\partial \vec{B}}{\partial t} \tag{2.3}$$

$$\nabla \times \vec{H} = \vec{J} + \frac{\partial \vec{D}}{\partial t} \tag{2.4}$$

In this work, we consider a linear, isotropic, and time-invariant medium. Its response is described by the permittivity ε, permeability μ, and electrical conductivity σ_e. The constitutive relations express $\vec{D}$, $\vec{B}$, and $\vec{J}$ in terms of the fields $\vec{E}$ and $\vec{H}$.

$$\vec{D} = \varepsilon \vec{E} \tag{2.5}$$

$$\vec{B} = \mu \vec{H} \tag{2.6}$$

$$\vec{J} = \sigma_e \vec{E} \tag{2.7}$$

2.2 One-Dimensional Model

In the one-dimensional model, we assume that the field varies only along the x-axis and that the wave propagates through a homogeneous cross-section. Derivatives with respect to the y and z coordinates are then formally equal to zero, so a grid is not required in those dimensions. This assumption corresponds to the propagation of a plane wave through layers normal to the x-axis.

$$\frac{\partial}{\partial y} = 0, \qquad \frac{\partial}{\partial z} = 0 \tag{2.8}$$

Because of this spatial restriction, the one-dimensional model cannot represent diffraction, scattering around an obstacle, or a complex transverse wave profile. Nevertheless, it accurately represents wave propagation, reflection, changes in velocity, and attenuation along a single axis. Each cell represents a thin layer with distinct electric and magnetic properties.

The domain length L is approximated by multiplying the integer number of cells K by the spatial step Δx. This relation provides the basis for the subsequent discretization of the one-dimensional model, while variations in grid width are later used in the performance analysis.

$$L = K\Delta x \tag{2.9}$$

2.3 The E_z and H_y Field Components

For the selected polarization, we retain the electric-field component E_z and the magnetic-field component H_y. Both components are perpendicular to the direction of propagation and to each other. The remaining four field components are set to zero.

$$\vec{E} = (0, 0, E_z), \qquad \vec{H} = (0, H_y, 0) \tag{2.10}$$

The ratio of the electric- and magnetic-field amplitudes in a lossless medium is determined by the wave impedance η. Its value depends on the local permittivity and permeability, so a change in material may produce reflected and transmitted waves. An abrupt change in impedance produces a stronger reflection than a gradual change.

$$\eta = \sqrt{\frac{\mu}{\varepsilon}} \tag{2.11}$$

Wave velocity also depends on the permittivity and permeability of the medium. The vacuum constants ε_0 and μ_0 correspond to the speed of light c_0. The program uses this reference velocity when determining the time step.

$$c = \frac{1}{\sqrt{\mu\varepsilon}} \tag{2.12}$$

2.4 System of Differential Equations

After introducing the one-dimensional polarization, we reduce Maxwell's curl equations to two coupled scalar equations. The first describes the change in H_y caused by the spatial variation of E_z, while the second describes the reciprocal effect of H_y on E_z and the attenuation caused by the electrical conductivity σ_e.

$$\frac{\partial H_y}{\partial t} = \frac{1}{\mu}\frac{\partial E_z}{\partial x} \tag{2.13}$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon}\frac{\partial H_y}{\partial x} - \frac{\sigma_e}{\varepsilon}E_z \tag{2.14}$$

A second-order wave equation can also be derived from this system. We differentiate the equation for E_z with respect to time and substitute the time derivative of H_y, assuming a homogeneous, lossless medium. The resulting equation relates wave velocity to the material properties and helps us select the resolution, although the FDTD method discretizes the original first-order system.

$$\frac{\partial^2 E_z}{\partial x^2} = \mu\varepsilon\frac{\partial^2 E_z}{\partial t^2} \tag{2.15}$$

2.5 Symmetric System of Equations

The equations from the previous section can be made symmetric by introducing a fictitious magnetic conductivity σ_m into Faraday's law. This additional term does not describe physical magnetic losses in a conventional material, but it allows the electric and magnetic equations to be written in the same form. We use this system to implement the absorbing boundary conditions in Chapter 4, while a more detailed derivation is provided in Appendix B.

The components E_z and H_y represent the electric and magnetic fields of the selected polarization. The permittivity ε and permeability μ describe the electric and magnetic properties of the medium, while σ_e denotes the electrical conductivity and σ_m the fictitious magnetic conductivity. Derivatives with respect to the spatial coordinate x couple the two field components, while the conductivity terms introduce their attenuation.

$$\frac{\partial H_y}{\partial t} = \frac{1}{\mu}\frac{\partial E_z}{\partial x} - \frac{\sigma_m}{\mu}H_y \tag{2.16}$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon}\frac{\partial H_y}{\partial x} - \frac{\sigma_e}{\varepsilon}E_z \tag{2.17}$$

Both conductivities are zero by default in the interior region of the simulation, so the symmetric system reduces to the basic lossless system. In the absorbing layers at the ends of the simulation domain, we specify matched electric and magnetic losses, thereby reducing wave reflection from the grid boundaries.

2.6 Material Properties

Material properties generally depend on position, which allows us to represent layered and inhomogeneous media. Relative permittivity ε_r specifies the ratio with respect to

vacuum, while relative permeability μ_r has the same role for the magnetic response. The absolute values are obtained by multiplying them by the corresponding physical constants.

$$\varepsilon = \varepsilon_r \varepsilon_0, \qquad \mu = \mu_r \mu_0 \tag{2.18}$$

A higher permittivity reduces the wave velocity if the permeability remains unchanged, while also changing the wave impedance. Electrical conductivity converts part of the field energy into heat and appears in the equation as an attenuation term. Magnetic conductivity is an auxiliary quantity used to match the electric and magnetic losses in the PML layer. The physical meanings and units of these quantities are summarized in Table 2.1.

Table 2.1: Material quantities in the theoretical model

Quantity	Physical meaning	Unit
ε	permittivity of the medium	F/m
μ	permeability of the medium	H/m
σ_e	electrical conductivity	S/m
σ_m	equivalent magnetic conductivity	adjusted unit

Spatial variations in these quantities change the local wave velocity and impedance, so reflected and transmitted components may arise at a material interface. Conductivity additionally determines how quickly the wave energy decays during propagation. In the next chapter, these quantities are distributed over the discrete spatial grid.

Chapter 3

Discretization of Maxwell's Equations

In the previous chapter, we reduced Maxwell's equations to a one-dimensional system for E_z and H_y. We now discretize these equations and obtain expressions that can be applied in the implementation. These expressions comprise the update equations and source injection.

3.1 The Finite-Difference Method and the Yee Grid

We replace a derivative with the ratio of a function's change to a small step, thereby transforming a continuous equation into a set of algebraic expressions that can be evaluated numerically. A central difference uses samples on both sides of the point under consideration and gives a second-order error for a sufficiently smooth function. Reducing the step decreases this error, but simultaneously increases the number of cells and the amount of computation.

$$\left(\frac{\partial f}{\partial x}\right)_i \approx \frac{f[i+1/2] - f[i-1/2]}{\Delta x} \tag{3.1}$$

In the standard FDTD method, the Yee grid provides the basis for the spatial arrangement of the fields [3]. By the convention adopted in this work, E_z is placed at integer spatial indices, while H_y is positioned between neighboring electric-field points. The same offset is applied in time, so the magnetic states occur halfway between the electric states. This arrangement allows us to calculate the central difference from neighboring values without additional interpolation.

$$x_E[k] = k\Delta x, \qquad x_H[k+1/2] = (k+1/2)\Delta x \tag{3.2}$$

$$t_E[n] = n\Delta t, \qquad t_H[n+1/2] = (n+1/2)\Delta t \tag{3.3}$$

The spatial and temporal arrangement of the components is shown in Figure 3.1. The fields are staggered in space and time, which allows them to be updated alternately. Each new magnetic state uses the known electric state, and each new electric state uses the newly completed magnetic state. This procedure is straightforward to implement and does not require solving a large system of equations.

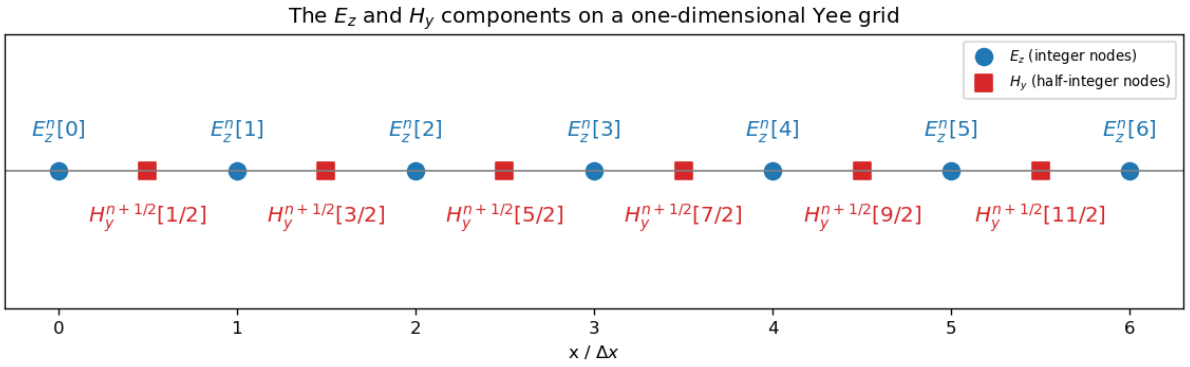


Figure 3.1: Positions of the E_z and H_y components on a one-dimensional Yee grid

3.2 Spatial and Temporal Discretization

We divide the spatial domain into K cells of equal width and the time interval into N consecutive steps. The indices k and n replace the continuous coordinates x and t, while Δx and Δt determine the resolution. A physical quantity is represented by an array of samples at discrete points.

$$x_k = k\Delta x, \qquad t_n = n\Delta t \tag{3.4}$$

The spatial derivative of E_z uses two neighboring electric-field points and belongs to the magnetic-field point between them. The derivative of H_y uses the magnetic-field points on either side of an electric-field point and therefore corresponds to an integer spatial index. These two expressions also determine the valid loop bounds in the program.

$$\left(\frac{\partial E_z}{\partial x}\right)_{k+1/2} \approx \frac{E_z^n[k+1] - E_z^n[k]}{\Delta x} \tag{3.5}$$

$$\left(\frac{\partial H_y}{\partial x}\right)_{k} \approx \frac{H_y^{n+1/2}[k+1/2] - H_y^{n+1/2}[k-1/2]}{\Delta x} \tag{3.6}$$

We also approximate the time derivatives with central finite differences, accounting for the temporal staggering of the components on the Yee grid. For the magnetic field, we use the values at times $n-1/2$ and $n+1/2$, so the approximation belongs to time n. For the electric field, we use the values at times n and $n+1$, and the approximation belongs to time $n+1/2$.

$$\left(\frac{\partial H_y}{\partial t}\right)^{n}_{k+1/2} \approx \frac{H_y^{n+1/2}[k+1/2] - H_y^{n-1/2}[k+1/2]}{\Delta t} \tag{3.7}$$

$$\left(\frac{\partial E_z}{\partial t}\right)^{n+1/2}_{k} \approx \frac{E_z^{n+1}[k] - E_z^{n}[k]}{\Delta t} \tag{3.8}$$

We assign the material properties to the spatial points of the corresponding field components. This aligns their arrangement with the Yee grid, so the permeability and magnetic conductivity are written as $\mu[k+1/2]$ and $\sigma_m[k+1/2]$, while the permittivity and electrical conductivity are written as $\varepsilon[k]$ and $\sigma_e[k]$. These quantities have no time index because the material properties do not change during the simulation.

3.3 Discrete System of Equations

Substituting the approximations from the previous section into the symmetric system of Maxwell's equations gives the discrete system of equations. In doing so, we follow the half-step conventions of the Yee grid.

In Faraday's law, we replace the time derivative of H_y and the spatial derivative of E_z with the corresponding central differences. We approximate H_y in the loss term by the arithmetic mean of its states at times $n-1/2$ and $n+1/2$. This gives the discrete magnetic-field equation at the point $k+1/2$.

$$\frac{H_y^{n+1/2}[k+1/2]-H_y^{n-1/2}[k+1/2]}{\Delta t}=\frac{1}{\mu[k+1/2]}\frac{E_z^n[k+1]-E_z^n[k]}{\Delta x} - \frac{\sigma_m[k+1/2]}{\mu[k+1/2]}\frac{H_y^{n+1/2}[k+1/2]+H_y^{n-1/2}[k+1/2]}{2}. \tag{3.9}$$

In the modified Ampère law, we similarly replace the time derivative of E_z and the spatial derivative of H_y. We approximate E_z in the loss term by the arithmetic mean of its states at times n and $n+1$. The resulting discrete equation belongs to the electric-field point k.

$$\frac{E_z^{n+1}[k]-E_z^n[k]}{\Delta t}=\frac{1}{\varepsilon[k]}\frac{H_y^{n+1/2}[k+1/2]-H_y^{n+1/2}[k-1/2]}{\Delta x} - \frac{\sigma_e[k]}{\varepsilon[k]}\frac{E_z^{n+1}[k]+E_z^n[k]}{2}. \tag{3.10}$$

These two equations form the discrete system for the electric and magnetic fields. Solving the first equation for $H_y^{n+1/2}[k+1/2]$ and the second for $E_z^{n+1}[k]$ gives the update equations presented in the next section.

3.4 The E_z and H_y Update Equations

We solve the discrete system for the field values at the next time step. For the magnetic field, this value is $H_y^{n+1/2}[k+1/2]$, and for the electric field it is $E_z^{n+1}[k]$. In this way, we obtain explicit update equations that are used directly in the sequential simulation algorithm implemented in Chapter 5.

We solve the discrete form of the modified Faraday law for $H_y^{n+1/2}[k+1/2]$ to obtain the magnetic-field update equation. The coefficient C_{h1} multiplies the magnetic-field value from the previous step, while C_{h2} multiplies the electric-field difference around the magnetic-field point $k+1/2$.

$$H_y^{n+1/2}[k+1/2]=C_{h1}[k+1/2]H_y^{n-1/2}[k+1/2]+C_{h2}[k+1/2]\left(E_z^n[k+1]-E_z^n[k]\right) \tag{3.11}$$

Similarly, we solve the discrete form of the modified Ampère law for $E_z^{n+1}[k]$ to obtain the electric-field update equation. The coefficient C_{e1} multiplies the electric-field value from the previous step, while C_{e2} multiplies the magnetic-field difference on either side of the electric-field point k.

$$E_z^{n+1}[k] = C_{e1}[k]E_z^n[k] + C_{e2}[k]\left(H_y^{n+1/2}[k+1/2] - H_y^{n+1/2}[k-1/2]\right) \tag{3.12}$$

The coefficients C_{h1}, C_{h2}, C_{e1}, and C_{e2} depend on the material parameters, conductivities, and the selected steps Δx and Δt, but not on the field values or the time index n. Their form and spatial dependence are examined in greater detail in the next section.

3.5 Update Coefficients and Material Modeling

In the previous section, we derived the update equations with electric and magnetic update coefficients. To write these coefficients more concisely, we introduce the dimensionless quantities a and b. The quantity a describes electric losses, while b describes magnetic losses; both are zero in a lossless region.

$$a[k] = \frac{\sigma_e[k]\Delta t}{2\varepsilon[k]}, \qquad b[k+1/2] = \frac{\sigma_m[k+1/2]\Delta t}{2\mu[k+1/2]} \tag{3.13}$$

The coefficients C_{e1} and C_{e2} belong to the electric-field points and contain the material quantities and grid steps. We calculate them before the time loop, eliminating division within the loop. Whenever a material parameter changes, these arrays are recalculated.

$$C_{e1}[k] = \frac{1 - a[k]}{1 + a[k]} \tag{3.14}$$

$$C_{e2}[k] = \frac{1}{\varepsilon[k]\,(1 + a[k])}\frac{\Delta t}{\Delta x} \tag{3.15}$$

The coefficients C_{h1} and C_{h2} have a symmetric form and belong to the magnetic-field points. After they have been prepared, the computational kernel uses only multiplication, addition, and subtraction on consecutive array elements. The same equations apply to an ordinary material, a conductive region, and the PML layer.

$$C_{h1}[k+1/2] = \frac{1 - b[k+1/2]}{1 + b[k+1/2]} \tag{3.16}$$

$$C_{h2}[k+1/2] = \frac{1}{\mu[k+1/2]\,(1 + b[k+1/2])}\frac{\Delta t}{\Delta x} \tag{3.17}$$

3.6 Excitation Pulse and Source Injection

A simulation with zero initial fields remains in that state until a source injects energy into the grid. A Gaussian pulse has a short effective duration and contains a broad range of frequencies, making it suitable for examining the transient response. The parameter n_0 specifies the time of the maximum, while w determines the temporal width of the pulse.

$$s_g[n] = \exp\left(-\frac{1}{2}\left(\frac{n_0 - n}{w}\right)^2\right) \tag{3.18}$$

A sinusoidal source is suitable for examining the steady-state response at a single selected frequency. Its behavior is determined by the amplitude A, frequency f, and initial

phase φ. The program allows multiple Gaussian and sinusoidal sources to be registered in the same simulation.

$$s_s[n] = A \sin(2\pi f n + \varphi) \tag{3.19}$$

The implementation uses soft source injection after updating E_z: the calculated value is not replaced, but the excitation is added to it. This procedure causes less disturbance to a wave that subsequently passes back through the source cell. The linearity of the model allows the contributions of multiple sources to be applied in sequence.

$$E_z^{n+1}[k_c] \leftarrow E_z^{n+1}[k_c] + s[n] \tag{3.20}$$

3.7 The Courant Condition and Time-Step Organization

The numerical stability of the FDTD method is determined by the dimensionless Courant number [2], [4]. The Courant number S relates the maximum wave propagation velocity c_0 to the ratio of the time step Δt to the spatial step Δx.

In the one-dimensional model, the Courant number must not exceed one, while the implementation uses the conservative value $S = 0.5$ by default. The time-step value follows directly from the selected Courant number and spatial step.

$$S = \frac{c_0 \Delta t}{\Delta x} \leq 1 \tag{3.21}$$

$$\Delta t = \frac{S \Delta x}{c} \tag{3.22}$$

We divide one time step into three phases: first we update H_y, then E_z, and finally apply all sources. After applying the sources, we increment the time counter by one and use the complete new state in the next step. The boundary between the first and second phases must remain explicit in the parallel version as well.

Chapter 4

The Perfectly Matched Layer

In the previous chapter, we derived the update equations for the electric and magnetic fields, but unwanted wave reflection may occur at the ends of the grid. To prevent this reflection, we place Bérenger's perfectly matched layer (PML) at each end of the grid to absorb the incident wave.

4.1 The Need for Absorbing Boundary Conditions

A numerical grid has a finite number of cells, even though the physical problem often represents a space with no apparent boundary. A simple grid termination behaves as a perfect electric conductor (PEC), from which the incident wave is reflected back into the simulation region. The reflected component then combines with the original wave and corrupts the useful result.

An absorbing boundary condition (ABC) reduces the wave energy before it reaches the end of the grid. One such boundary condition is Bérenger's PML [5], which represents a lossy region matched to the adjacent medium. In the one-dimensional model, we place this region at the left and right ends of the domain.

Practical absorption is not completely perfect because of finite spatial and temporal discretization. Its quality depends on the number of cells, the conductivity profile, and the shortest wavelength in the signal. The layer parameters must therefore provide gradual attenuation before the wave reaches the final grid point.

4.2 The Basic Principle of the PML Method

Bérenger's PML is an absorbing layer that surrounds the physical simulation region with an artificial lossy material whose impedance is matched to that of the interior medium. The matched impedance reduces reflection at the interface, while the increasing losses attenuate the wave as it passes through the layer.

We set the conductivity to zero at the interface with the interior region and then increase it gradually toward the outer edge of the domain, as shown in Figure 4.1. The wave therefore does not encounter an abrupt change in the first PML cell; instead, its amplitude decreases as it passes through multiple cells. Only a small fraction of the initial energy reaches the outer edge.

The left and right layers use the same profile in opposite directions. The central part of the grid remains free of additional conductivity and represents the physical simulation

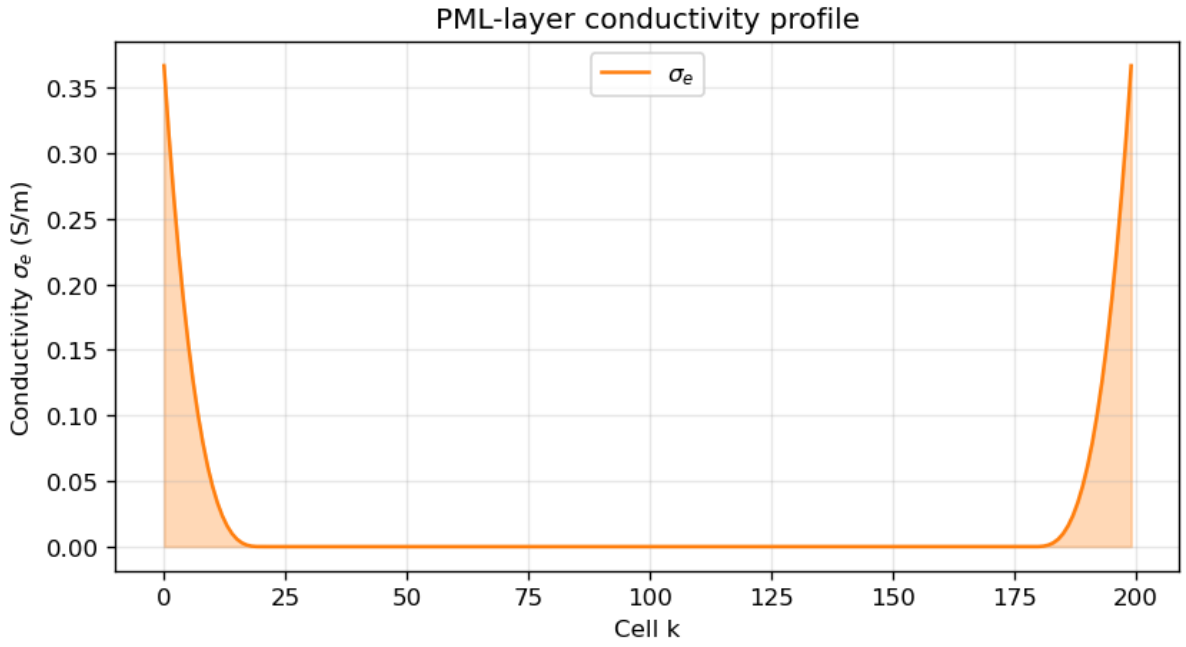


Figure 4.1: Electrical conductivity in the left and right PML layers

region. Increasing the PML thickness reduces the physical region but improves the effectiveness of wave absorption.

4.3 The Matching Condition

We select the PML parameters so that its wave impedance matches the impedance of the adjacent medium. Unequal attenuation of the electric and magnetic fields changes their ratio and creates an impedance discontinuity. Such a discontinuity would cause partial wave reflection at the interface between the physical region and the absorbing layer [5].

In the symmetric system given by equations (2.16) and (2.17), the electrical conductivity σ_e determines the attenuation of the electric field, while the magnetic conductivity σ_m determines the attenuation of the magnetic field. Their effects also depend on the permittivity ε and permeability μ, respectively, so the matching condition is

$$\frac{\sigma_m}{\mu} = \frac{\sigma_e}{\varepsilon} \tag{4.1}$$

This condition provides equal relative attenuation of both field components and prevents their ratio from changing as the wave enters the layer. Consequently, no reflection occurs at the interface in the continuous model, while the wave energy is gradually absorbed within the PML. A small reflection remains in the discrete model, but it is further reduced by a gradual conductivity profile and correct sampling on the Yee grid.

4.4 PML Parameters and Coefficients

The physical PML thickness d equals the product of the number of PML cells N_{PML} and the spatial step Δx. The maximum electrical conductivity $\sigma_{\max}$ depends on this

thickness, the target reflection coefficient R_0, and the local wave impedance η from equation (2.11). A thicker layer permits a slower increase in conductivity for the same final attenuation.

In the standard PML implementation, the target reflection coefficient is $R_0 = 10^{-6}$ and the polynomial grading order is $m = 3$ [5]. The cubic profile provides a smooth transition into the layer and considerably stronger attenuation near the outer edge.

$$d = N_{\mathrm{PML}} \Delta x \tag{4.2}$$

$$\sigma_{\max} = -\frac{(m+1)\ln R_0}{2\eta d} \tag{4.3}$$

We define the electrical conductivity σ_e using a polynomial profile and obtain the magnetic conductivity σ_m from the matching condition. The electric and magnetic depths are sampled at positions offset by half a cell. The normalized depth q is zero in the interior and approaches one at the outer edge.

$$\sigma_e(x) = \sigma_{\max} q(x)^m \tag{4.4}$$

$$\sigma_m(x) = \sigma_e(x) \frac{\mu(x)}{\varepsilon(x)} \tag{4.5}$$

These parameters are fixed in the current implementation, while the user selects only the number of PML cells through the public interface. Fixed values simplify usage and enable comparable simulation experiments, while a different profile can be introduced later without changing the computational kernel.

The absorbing PML affects only the conductivities σ_e and σ_m, which are treated as part of the simulation material. These conductivities affect the loss coefficients a and b, which in turn affect the update coefficients C_{h1}, C_{h2}, C_{e1}, and C_{e2}. The structure of the finite-difference method remains unchanged, and the update equations remain the same.

Chapter 5

Software Implementation

In the preceding chapters, we derived the FDTD update equations and introduced the PML. This chapter presents the main parts of the software implementation written in the C programming language.

5.1 Software Architecture

We divide the implementation into smaller components for the state, fields, materials, coefficients, sources, and acceleration mode. Each component represents one part of the mathematical model that can be verified independently. This division establishes a clear connection between the physical model and the program code.

The `Fdtd_EzSim` structure connects all parts of a simulation. The grid size, fields, materials, coefficients, and sources are nevertheless stored in separate structures. The main software components and their roles are shown in Table 5.1.

Table 5.1: Main components of the software solution

Component	Structure	Role
state	`Fdtd_EzState`	parameters K, S, Δx, and Δt
fields	`Fdtd_EzFields`	arrays for E_z and H_y
material	`Fdtd_EzMaterial`	arrays for ε, μ, σ_e, and σ_m
coefficients	`Fdtd_EzCoeffs`	arrays for C_{e1}, C_{e2}, C_{h1}, and C_{h2}
sources	`Fdtd_EzSources`	excitation functions and their data
simulation	`Fdtd_EzSim`	connection between all components

This architecture allows the serial and parallel kernels to share the same data and public interface. Changing the acceleration mode therefore does not modify the material, sources, or result format. Such separation is important for a fair performance comparison because both versions execute the same numerical procedure.

5.2 The Static Library and Public API

The software is delivered as a package consisting of the `libfdtd.a` static library, a set of headers, two executable programs, and utility scripts. Users can construct simulations through the library functions. The `fdtd` executable provides access to the same functions from the command line.

The public `fdtd.h` header includes the types, constants, and functions required by the user. Function names begin with the `Fdtd_` prefix, reducing the possibility of conflicts with other C libraries. A separate `fdtd_ez.h` header groups the components belonging to the selected polarization.

```
#include <fdtd.h>

Fdtd_EzSim *sim = Fdtd_EzSimCreate(200);
Fdtd_EzSimSetPML(sim, 20);
Fdtd_EzSimRun(sim, 100);
Fdtd_EzSimDestroy(sim);
```

The basic lifecycle comprises creating, configuring, running, and destroying the object. Between these steps, the user can change the material, spatial step, Courant number, or acceleration mode. A clear lifecycle reduces the possibility of memory leaks and facilitates reproducible experiments.

5.3 Simulation Representation and Data Structures

We represent each spatial quantity as a separately allocated dynamic array of K real numbers. Consecutive elements of an array use cache lines efficiently and preserve favorable spatial locality. The `Fdtd_EzFields` structure is one example and stores the electromagnetic-field values on the grid.

```
typedef struct {
    size_t ff_width;       // Number of cells
    Fdtd_FloatType *ff_hy; // Magnetic field H_y
    Fdtd_FloatType *ff_ez; // Electric field E_z
} Fdtd_EzFields;
```

The simulation-state structure stores the width, Δx, Δt, Courant number, current time, and selected acceleration mode. The time step is recalculated automatically whenever the spatial step or Courant number changes. This approach prevents every user program from having to maintain the important stability relationship itself.

```
typedef struct {
    size_t fs_width;             // Number of cells in the simulation
    Fdtd_FloatType fs_dx;        // Spatial step (m)
    Fdtd_FloatType fs_dt;        // Time step (s)
    Fdtd_FloatType fs_courant;   // Courant number S = c * dt / dx
    Fdtd_FloatType fs_time;      // Current simulation time (number of steps)
    Fdtd_AccelType fs_accel;     // Acceleration mode (serial / OpenMP)
} Fdtd_EzState;
```

Material properties and update coefficients are stored in four arrays each, with the same number of elements as the field arrays. This choice uses more memory than calculating the coefficients on demand, but considerably reduces the number of operations in every time step. The same coefficients are used hundreds of times, so the lower computational cost justifies the additional memory.

5.4 FDTD Algorithm Implementation

We implement the update equations from Chapter 3 with two loops in the `Fdtd_KernelSerial_EzSimRun()` function. Local pointers shorten access through the structures, while the `restrict` qualifier informs the compiler that the arrays do not overlap. The resulting code remains readable and suitable for automatic optimization.

```
// FDTD algorithm parameters
Fdtd_FloatType * restrict Hy = sim->fsim_fields->ff_hy;
Fdtd_FloatType * restrict Ez = sim->fsim_fields->ff_ez;
Fdtd_FloatType * restrict Ch1 = sim->fsim_coeffs->fc_ch1;
Fdtd_FloatType * restrict Ch2 = sim->fsim_coeffs->fc_ch2;
Fdtd_FloatType * restrict Ce1 = sim->fsim_coeffs->fc_ce1;
Fdtd_FloatType * restrict Ce2 = sim->fsim_coeffs->fc_ce2;
size_t width = sim->fsim_state->fs_width;

// FDTD algorithm
for (size_t n = 0; n < nsteps; n++) {

    // Update Hy
    for (size_t k = 0; k < width - 1; k++) {
        Hy[k] = Ch1[k] * Hy[k]
              + Ch2[k] * (Ez[k + 1] - Ez[k]);
    }

    // Update Ez
    for (size_t k = 1; k < width; k++) {
        Ez[k] = Ce1[k] * Ez[k]
              + Ce2[k] * (Hy[k] - Hy[k - 1]);
    }

    // Inject Ez source and advance time
    Fdtd_EzSourcesApply(sim->fsim_sources, sim);
    Fdtd_EzStateNextStep(sim->fsim_state);
}
```

The first loop updates $H_y^{n+1/2}[k+1/2]$ for indices from zero through $K-2$ because it reads two neighboring values of $E_z^n[k]$. The second loop starts at index one and updates $E_z^{n+1}[k]$ through the end of the domain while reading $H_y^{n+1/2}[k-1/2]$. The bounds of both loops follow directly from the field positions on the Yee grid.

A dependency exists between the two loops, but the iterations within each loop are independent. The serial code serves as the reference implementation for validating the

parallel version and calculating speedup. Users do not invoke the simulation kernel directly, but instead call the public `Fdtd_EzSimRun()` function.

5.5 PML Integration

The `Fdtd_EzMaterialSetPML()` function fills the electrical- and magnetic-conductivity arrays at both ends of the domain. It is called by the public `Fdtd_EzSimSetPML()` function, which subsequently recalculates the update coefficients through `Fdtd_EzCoeffsUpdate()`. In this way, configuring the PML changes the update-coefficient values used by the FDTD algorithm.

The `pml_width` argument specifies the number of PML cells N_{PML}, while `dx` is the spatial step Δx; multiplying these two quantities gives `thickness`, the PML thickness d. The helper function `Fdtd_PmlDepth()` calculates the normalized depth q. Selecting `order = 3` gives the cubic conductivity profile $m = 3$, while `refl` represents the target reflection coefficient R_0.

```
// PML-layer parameters
int order = 3;
Fdtd_FloatType refl = 1.0e-6;
Fdtd_FloatType thickness = pml_width * dx;
```

The loop skips the interior and processes only the cells in the left and right PML regions. For each cell, it calculates the maximum conductivity `sigma_max` from the local impedance `eta`, then determines separate depths for the electric- and magnetic-field points. This produces spatially staggered `sigma_e` and `sigma_m` profiles that subsequently affect the update coefficients.

```
// Grade conductivity in the left and right PML layers
for (size_t k = 0; k < width; k++) {
    if (k > pml_width && k < width - pml_width)
        continue;

    // Wave impedance and maximum conductivity for the given reflection
    Fdtd_FloatType mu = material->fm_mu[k];
    Fdtd_FloatType eps = material->fm_eps[k];
    Fdtd_FloatType eta = sqrt(mu / eps);
    Fdtd_FloatType sigma_max =
        -(Fdtd_FloatType) (order + 1) * log(refl)
        / (2.0 * eta * thickness);

    // Magnetic conductivity at the H point (x = k + 1/2),
    // condition sigma*/mu = sigma/eps
    Fdtd_FloatType dh = Fdtd_PmlDepth(
        (Fdtd_FloatType) k + 0.5, pml_width, width);
    if (dh > 0.0) {
        material->fm_sigma_m[k] =
            sigma_max * pow(dh, (Fdtd_FloatType) order) * mu / eps;
    }
```

```
    // Electric conductivity at the E point (x = k)
    Fdtd_FloatType de = Fdtd_PmlDepth(
        (Fdtd_FloatType) k, pml_width, width);
    if (de > 0.0) {
        material->fm_sigma_e[k] =
            sigma_max * pow(de, (Fdtd_FloatType) order);
    }
}
```

5.6 Memory Management

The large number of consecutive accesses makes memory layout an important aspect of performance. The `Fdtd_AllocArray()` function allocates aligned storage, using 64-byte alignment by default, and verifies that the allocation succeeded. Alignment facilitates vector loads and reduces the number of unfavorable accesses that cross cache-line boundaries.

Every function that creates a structure has a corresponding function that destroys it. The main simulation object destroys its components in the reverse order from their creation. The user therefore needs to call only `Fdtd_EzSimDestroy()`, without manually freeing the internal arrays.

We initialize the fields to zero when they are created or reset, so repeated measurements begin from the same state. Before each sample, the benchmark performs a short warm-up, preventing the first access to allocated pages from being included in the measured time. This procedure reduces the effect of memory initialization on comparisons between computational kernels.

5.7 Simulation Configuration and Execution

The command-line tool allows the model to be configured without writing a new program. Users can specify the number of cells, spatial step, Courant number, material regions, boundary, source, and number of time steps. This configuration is convenient for quickly repeating simulation examples.

```
fdtd --set-width=200 --set-pml=20 \
     --add-pulse --run=100 results/gauss.csv
```

The program first processes the options, then creates the simulation and applies the requested material changes. Sources are registered after the materials, and the time loop begins only after all coefficients are ready. The result is finally written to a comma-separated values (CSV) file containing the state of every cell.

An existing state can be loaded with the `--input-csv` option and advanced by a new number of steps. This capability is useful when a long computation is divided into several parts or when the same instant is used as the basis for multiple experiments. The `--get-dt` option additionally provides a direct check of the calculated time step.

5.8 Visualization and Performance Measurement

Raw arrays of numbers make it difficult to determine whether a wave propagates as expected, so the project includes the `plot_sim.py` tool. It reads a CSV state and plots the fields, material quantities, coefficients, and absorbing-layer profile. The plots thereby connect the program's internal data with the physical meaning of the model.

A separate `benchmark` program measures the serial reference and OpenMP version with different thread counts. For every repetition, it records the execution mode, number of threads, grid width, number of steps, and measured duration. The `plot_benchmark.py` script calculates throughput, speedup, and efficiency from these values and produces plots that compare domain sizes.

Simulation output is kept separate from performance results, so the time spent plotting or writing a large state does not affect the measured kernel. The measured interval includes only the time-loop call with data that has already been prepared. The resulting values therefore describe the computational portion that is actually parallelized.

5.9 State Format and Simulation Resumption

The output CSV file contains simulation parameters in comments and one tabular row for every cell. The header records the width, spatial step, Courant number, and current time, while the columns contain the fields, relative material properties, conductivities, and coefficients. This format is human-readable and can be loaded directly into commonly used analysis tools.

```
# width: 200
# set_dx: 0.01
# set_courant: 0.5
# set_time: 100
k,Hy,Ez,mu_r,eps_r,sigma_m,sigma_e,ch1,ch2,ce1,ce2
```

The `Fdtd_EzSimCreateFromCSV()` function first reads the width and creates a new object of the appropriate size. It then loads the steps, time, fields, and material arrays before recalculating the coefficients. We recalculate the coefficients to ensure that they correspond to the parameters that were actually loaded.

Saving the complete state makes it possible to resume a long simulation and separate computation from visualization. The cost of this approach is a large text file and slower output compared with a binary format. Output is performed outside the performance-measurement interval and only when the user requests a result.

5.10 Input Validation and Error Handling

The command-line tool checks whether the required parameters are present and whether a recognized acceleration mode was selected. At most 32 sources and the same number of material modifications can be specified, preventing writes beyond the bounds of the designated arrays. The range of every material modification is also limited by the actual grid width.

Memory-allocation functions use `assert()` to terminate the program immediately if the system cannot provide the requested storage. The same type of check is used when

opening a file and reading the width from a saved state. This behavior is sufficient for a research prototype because it prevents execution from continuing with an invalid pointer or incomplete object.

A more robust library intended for wider use should return an error code instead of terminating the process. It should also verify that the PML width, source position, Courant number, and all numerical values are within their valid ranges. Such an extension would allow the calling program to display a meaningful message and safely release any resources that have already been allocated.

5.11 Build Process and Distribution Structure

The project uses a `Makefile`, so the library, programs, and tools are always built with the same options. The `make build` command compiles the source files, creates the static library, and links the user-facing programs. Details of the project structure itself are provided in Appendix D.

```
dist/
|-- bin/
|   |-- benchmark
|   |-- fdtd
|   `-- tools/
|-- docs/
|   `-- Fdtd_Thesis.pdf
|-- include/
|-- lib/
|   `-- libfdtd.a
`-- buildinfo
```

The `dist` directory stores the executable programs, public headers, library, utility tools, documentation, and build information separately. Users do not need the internal source-tree structure to link their programs with `libfdtd.a`. The plotting and measurement tools are available in the same directory, eliminating the need to collect files manually.

Chapter 6

Parallelization with OpenMP

In the preceding chapter, we described the structure of the serial implementation. This chapter analyzes which parts of the algorithm can be executed in parallel using OpenMP.

6.1 Parallel Execution Model

Modern processors have multiple cores, but a serial program uses only one of them at a time. OpenMP makes it possible to distribute the iterations of a regular loop among threads that share the same address space [6]. This approach is well suited to the FDTD algorithm because all threads require the same field and coefficient arrays.

When the program enters an `omp parallel` region, the initial thread creates a team of worker threads. The region encloses the entire sequence of time steps, so the same team remains active until the parallel kernel call finishes. This avoids repeatedly creating and destroying the team for every spatial loop or time step.

This organization is illustrated in Figure 6.1 as the OpenMP fork-join model. The initial thread creates the team once upon entering the parallel region, the threads then process their assigned spatial blocks throughout all time steps, and the team joins only after the parallel kernel has finished.

Within the active region, the `omp for` directive distributes the iterations of the spatial loops, while `omp single` assigns source application and advancement of the time counter to one thread. The compiler and runtime library manage work distribution and synchronization. The numerical expressions remain identical to those in the serial version.

The `schedule(static)` clause instructs the scheduler to divide the iterations into approximately equal, fixed blocks and assign them to the threads. This distribution is appropriate for the FDTD algorithm because each iteration of a spatial loop requires approximately the same amount of work.

6.2 Dependency Analysis and Work Distribution

Parallelization is correct only if different threads do not write to the same destination without an established ordering. In the H_y loop, iteration k reads $E_z^n[k]$, $E_z^n[k+1]$, and the old $H_y^{n-1/2}[k+1/2]$, while writing only the new $H_y^{n+1/2}[k+1/2]$. Because the destinations are distinct, all iterations of this loop can execute independently.

$$\mathrm{Read}_H[k] = \{E_z^n[k], E_z^n[k+1], H_y^{n-1/2}[k+1/2]\} \tag{6.1}$$

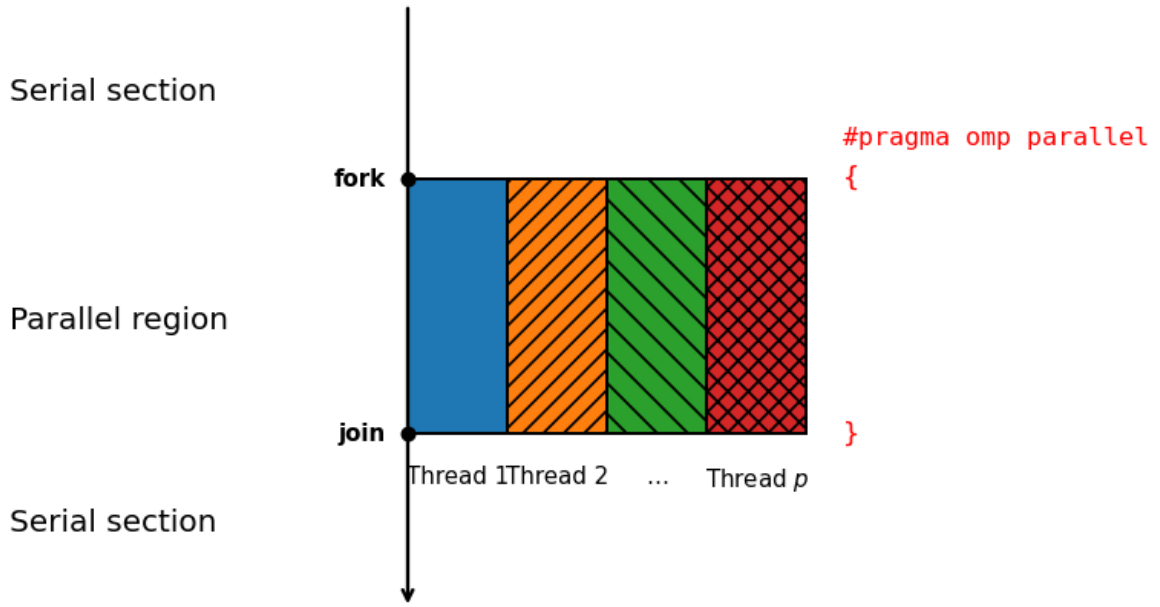


Figure 6.1: Thread fork and join in the OpenMP fork-join model

$$\text{Write}_H[k] = \{H_y^{n+1/2}[k+1/2]\} \tag{6.2}$$

The E_z loop has the same property: each iteration reads two neighboring $H_y^{n+1/2}$ elements and writes only $E_z^{n+1}[k]$. Overlapping read sets are not a problem because multiple threads may simultaneously read the same unchanged value. What matters is that the write sets of two different iterations do not overlap.

$$\text{Read}_E[k] = \{H_y^{n+1/2}[k-1/2], H_y^{n+1/2}[k+1/2], E_z^n[k]\} \tag{6.3}$$

$$\text{Write}_E[k] = \{E_z^{n+1}[k]\} \tag{6.4}$$

The iterations are independent within each phase, but the magnetic and electric phases are not independent of one another. Every iteration of the second loop must observe the already updated neighboring H_y values. Parallelization therefore partitions space while preserving the temporal ordering of the phases.

6.3 Parallelization of the FDTD Algorithm

In the parallel kernel, a single `omp parallel` region encloses the outer time-step loop. The `default(none)` clause requires the data-sharing attribute of every variable to be specified explicitly. The pointers to the field and coefficient arrays, the grid width, and the number of steps are declared `firstprivate`, while the `sim` object is shared.

```
#pragma omp parallel \
    default(none) \
    firstprivate(Hy, Ez, Ch1, Ch2, Ce1, Ce2, width, nsteps) \
```

```
    shared(sim)
{
    for (size_t n = 0; n < nsteps; n++) {

        // Update Hy
        #pragma omp for schedule(static)
        for (size_t k = 0; k < width - 1; k++) {
            Hy[k] = Ch1[k] * Hy[k]
                  + Ch2[k] * (Ez[k + 1] - Ez[k]);
        }

        // Update Ez
        #pragma omp for schedule(static)
        for (size_t k = 1; k < width; k++) {
            Ez[k] = Ce1[k] * Ez[k]
                  + Ce2[k] * (Hy[k] - Hy[k - 1]);
        }

        // Inject Ez source
        #pragma omp single
        {
            Fdtd_EzSourcesApply(sim->fsim_sources, sim);
            Fdtd_EzStateNextStep(sim->fsim_state);
        }
    }
}
```

The pointer copies received by the threads still refer to the same shared arrays. Both spatial loops use `omp for schedule(static)` because all iterations perform approximately the same number of operations. Each thread is consequently assigned a contiguous block of indices, preserving favorable spatial locality.

The simulation function selects the serial or parallel mode according to the value of `fs_accel`. In parallel mode, one thread processes the sources and time counter in the `omp single` block while the other threads wait for it to finish. The same object can therefore be used for reference and parallel measurements without reconstructing the model.

6.4 Synchronization and Thread Management

By default, an `omp for` directive places a barrier at the end of the distributed loop. The first barrier ensures that all H_y values have been updated before any thread begins calculating E_z. The second barrier completes the entire E_z update before one thread applies the source.

$$H_y^{n+1/2} \longrightarrow \text{barrier} \longrightarrow E_z^{n+1} \longrightarrow \text{barrier} \longrightarrow \text{source and time} \tag{6.5}$$

An `omp single` block also has an implicit barrier at its end. It prevents the threads from starting the next time step before the source has been applied and the time counter incremented. Each step therefore contains three necessary synchronization points, but the parallel region is opened only once for the entire kernel call.

Removing the first barrier with the `nowait` clause would be numerically incorrect because a faster thread could read old values from a slower thread's block. The benchmark program sets the number of threads using `omp_set_num_threads()`, and each measurement executes the same number of time steps. The measured speedups can therefore be compared directly.

6.5 Memory Access Locality

Each spatial loop traverses the arrays in increasing index order, so after one element it soon uses neighboring elements from the same cache line. This spatial locality reduces the number of main-memory accesses compared with a random access pattern. Static scheduling preserves the same favorable pattern for every thread.

The arrays are much larger than the processor registers, and the largest grids cannot fit entirely in cache. At every time step, the fields and coefficients are read and the two fields are written back. Because the kernel performs few arithmetic operations per transferred byte, it can become limited by memory bandwidth.

False sharing occurs when two threads write to different elements that belong to the same cache line. With large static blocks, this effect is confined to their boundaries and represents only a small fraction of the total work. Aligned arrays and sufficiently large blocks further reduce its impact.

6.6 Speedup Limitations

Amdahl's law provides an upper bound on speedup, showing that the serial portion of a program limits overall acceleration regardless of the number of threads [7]. In this implementation, coefficient preparation, application of a small number of sources, and time-step management remain serial. In addition, three synchronization points in every step introduce overhead that is absent from serial execution.

$$S_p \leq \frac{1}{(1-\alpha) + \frac{\alpha}{p}} \tag{6.6}$$

The parameter α denotes the fraction of execution time suitable for parallel execution, while p is the number of threads. On a small grid, the useful work per thread is brief, so the cost of creating the team and repeatedly reaching barriers can eliminate the benefit entirely. As the grid grows, the ratio of useful work to synchronization overhead becomes more favorable.

On large grids, speedup begins to be limited by the shared memory subsystem. Adding threads increases the demand for bandwidth but does not increase the amount of computation performed per item of data. In addition to speedup, we therefore consider parallel efficiency, which indicates how effectively each additional thread is used.

$$E_p = \frac{S_p}{p} \tag{6.7}$$

6.7 Arithmetic Intensity of the FDTD Algorithm

In the one-dimensional FDTD algorithm, each cell update performs a small number of additions and multiplications, but it reads a field value and two coefficients and writes the field back. The ratio of arithmetic work to transferred data is therefore relatively low compared with procedures that reuse the same values in registers. This ratio is known as arithmetic intensity and helps predict when memory bandwidth will become the limiting factor.

$$I = \frac{F}{Q} \tag{6.8}$$

The quantity F denotes the number of arithmetic operations, while Q is the amount of data transferred between the processor and the memory hierarchy. The exact value depends on cache behavior, the write policy, and the `Fdtd_FloatType` type, so it cannot be obtained merely by counting operations in the source code. The loop structure nevertheless shows that every additional thread uses the same shared memory bandwidth. Table 6.1 summarizes the main reads and writes of both kernels.

Table 6.1: Data accesses in one iteration

Kernel	Main reads	Main write
H_y	old H_y, two E_z values, C_{h1} and C_{h2}	new H_y
E_z	old E_z, two H_y values, C_{e1} and C_{e2}	new E_z

Good spatial locality allows neighboring reads to share a cache line, but it does not eliminate the need to fetch large arrays from memory. With sufficiently many threads, bandwidth is therefore expected to saturate before the arithmetic units are fully utilized. The measured increase in throughput should be interpreted in light of this limitation.

6.8 Choosing Between Serial and Parallel Execution

Parallel execution is not the best choice in every situation, as the results for small grids later confirm. The library currently leaves it to the user to select `FDTD_ACCEL_NONE` or `FDTD_ACCEL_OPENMP`. This decision is clear and predictable, but it requires the user to understand the size and environment of the problem.

A simple automatic choice could use a grid-width threshold obtained from a short initial measurement. If the parallel version does not reduce execution time over several trial steps, the simulation could continue in serial mode. The threshold should not be permanently tied to a particular number of cells because it depends on the processor, the number of threads, and the state of the system.

Another approach is to let the user select the behavior explicitly while the library merely issues a warning for an obviously small domain. This preserves the reproducibility of scientific measurements and avoids a hidden change in execution mode. Automatic tuning may be more practical for production use, whereas manual control is better suited to experimental work.

Chapter 7

Experimental Methodology

In the preceding chapter, we presented the parallel version of the FDTD algorithm. We now define how its performance is measured and compared with that of the serial version.

7.1 Hardware and Software Environment

All performance measurements were performed on the `rtidev5.etf.rs` computer, using an Intel Core i7-11700F processor with eight physical and sixteen logical cores. The availability of two hardware threads per core allows execution with up to eight threads and the use of simultaneous multithreading above that limit to be examined separately. Details of the hardware and software environment are given in Table 7.1.

Table 7.1: Measurement environment recorded with the results

Property	Value
computer	rtidev5.etf.rs
processor model	11th Gen Intel Core i7-11700F @ 2.50 GHz
physical cores	8
logical cores	16
threads per core	2
L2 cache	4 MiB, eight instances
L3 cache	16 MiB, one instance
compiler	GCC 11.4.0
compiler options	`-O3 -march=native -fopenmp -Wall`
measurement date	August 15, 2026

The program was compiled with a high optimization level and OpenMP support. The `-march=native` option permits the use of instructions available on the particular processor, while `-O3` enables aggressive loop optimization. The same options are used for serial and parallel execution, making their comparison fair.

The program automatically writes system information as comments at the beginning of the resulting CSV file. The same header records `OMP_DYNAMIC=false`, the placement policy `OMP_PLACES=cores`, and the binding policy `OMP_PROC_BIND=close`. The plotting script skips comment lines, but they remain available for subsequent verification.

7.2 Simulation Parameters

The measurements cover domains whose widths double from 10,000 to 1,280,000 cells because the relationship between computational work and memory traffic changes with the size of the working set. Each run performs 1,024 time steps. This choice provides a sufficiently long measurement interval and makes adjacent domain sizes easy to compare. Table 7.2 summarizes all measurement parameters.

Table 7.2: Basic measurement parameters

Parameter	Values
width K	10,000; 20,000; 40,000; 80,000; 160,000; 320,000; 640,000; 1,280,000
number of steps N	1,024
number of threads p	1 to 16
number of repetitions	10
PML width	20 cells at each boundary
source	Gaussian pulse at the center

Each test simulation has a PML twenty cells wide and a Gaussian source at the center of the grid. The pulse parameters are $n_0 = 40$ and $w = 12$, corresponding to the example used to verify physical behavior. Although the source adds a small amount of serial work, it ensures that the measured loop represents realistic use of the library.

Serial execution is measured separately, while parallel execution is measured for every thread count from one through sixteen. A parallel measurement with one thread isolates the OpenMP overhead without benefiting from an additional core. Values above eight threads show whether the second hardware thread on each physical core provides any further benefit.

7.3 Performance Measurement Methodology

Before each group of measurements, the simulation is repeatedly reset and run until the warm-up lasts for at least 0.1 seconds. This activates memory pages, loads the code into cache, and starts the OpenMP runtime before measurement. Each measurement then begins by resetting the same state again.

Time is measured with `omp_get_wtime()` immediately before and after the call to `Fdtd_EzSimRun()`. Simulation creation, coefficient preparation, file opening, and plot generation are excluded from the measured interval. This isolates the time loop that is the subject of parallelization.

Each combination of grid width and thread count is repeated ten times because the operating system, processor frequency, and cache state affect execution time. The median is used as the representative value for each group, and the reference time for speedup is the median of the serial measurements for the same width. Every individual row is retained in the CSV file for subsequent verification.

7.4 OpenMP Environment Control

The number of worker threads is set before each group of measurements with `omp_set_num_threads()`. Ten samples are then collected with the same number of threads before moving to the next value. This order reduces the number of environment changes and simplifies interpretation of the output table.

Iteration distribution in both kernels uses `schedule(static)`, so each thread receives an approximately equal contiguous block. Because every iteration contains the same numerical expression, no imbalance that would justify dynamic scheduling is expected. Static scheduling also has low management overhead and a predictable memory-access pattern.

The thread count is not increased beyond the sixteen logical cores recorded in the environment. Up to eight threads, each thread can use a separate physical core; higher values activate the second hardware thread on some of the cores. Results above eight threads must therefore also be interpreted in terms of shared execution and memory resources within each core.

7.5 Performance Metrics

The primary measured quantity is the time T_p required to perform N steps on a grid of width K with p threads. Time alone is insufficient for comparing different widths because a larger grid performs more useful work. We therefore also calculate throughput in Mcells/s.

$$P_p = \frac{KN}{10^6 T_p} \tag{7.1}$$

Speedup S_p is the ratio of the median serial execution time $\tilde{T}_{\text{seq}}$ to the execution time of the parallel version being considered. A value greater than one indicates that parallelization provides a benefit, whereas a value below one indicates a slowdown. Ideal speedup equals the number of threads, but is not expected in a real program because of serial work, synchronization, and shared hardware resources.

$$S_p = \frac{\tilde{T}_{\text{seq}}}{T_p} \tag{7.2}$$

Efficiency E_p divides speedup by the number of threads and indicates the utilization of the additional processing resources. A value close to one represents nearly linear speedup, while a lower value indicates growing overhead or saturation of a shared resource. Considering throughput, speedup, and efficiency together provides a more complete picture than any single metric.

$$E_p = \frac{S_p}{p} \tag{7.3}$$

7.6 Verification of Numerical Correctness

A faster program is of no value if it changes the physical result, so the parallelization uses the same expressions and loop bounds as the reference implementation. Different

threads write to distinct elements, while the implicit barrier completes the H_y phase before the E_z phase begins. This preserves the same ordering of dependent operations in every time step.

Field verification includes the positions of the two parts of the Gaussian pulse, the opposite sign of the magnetic component, and attenuation within the PML. Plots of the material parameters additionally confirm that losses are present only at the ends of the domain. These tests can reveal an indexing error that would not be apparent from execution time alone.

The same simulation was executed in serial mode and in OpenMP mode with four threads, after which the complete output CSV files were compared. After one hundred time steps, all parameters and field values were identical. This comparison confirms that the parallel distribution did not alter the numerical result.

7.7 Statistical Processing of Repeated Measurements

Repeated measurements are not completely identical because the operating system occasionally interrupts the program, while processor frequency and cache state vary. For each grid width, execution mode, and thread count, the ten obtained values are sorted. The median is calculated as the mean of the fifth and sixth samples in this ordered sequence.

The plots show median execution times from which throughput, speedup, and efficiency are derived. This statistic is less sensitive to an occasional slow execution than the arithmetic mean and therefore describes a typical run more accurately. The same median serial execution time serves as the reference for all OpenMP configurations at a given width.

The original CSV remains part of the project and contains all 1,360 measured rows, allowing the dispersion and alternative statistical measures to be examined later. The plots do not show intervals of variation, so small differences between adjacent thread counts should not be interpreted as a certain advantage. Nevertheless, ten repetitions provide a more stable basis than a single isolated measurement.

7.8 Threats to Experimental Validity

Internal validity depends on whether the measured time actually belongs to the computational kernel. Timed warm-up, state resetting, and narrowly placed timestamps reduce the effect of initialization and result output. Operating-system activity and dynamic changes in clock frequency may nevertheless affect individual measurements.

External validity is limited by the use of a single processor model, one compiler, and at most sixteen threads. Another architecture may have different cache sizes, vector instructions, and memory bandwidth, so the exact break-even points cannot be considered universal. The measurement procedure is nevertheless reproducible and can be applied to another machine without changing the metrics.

Construct validity depends on the chosen throughput metric, which counts one cell once per time step even though there are two spatial loops. The CSV records only the measured time in seconds, while the script consistently calculates throughput in Mcells/s from K, N, and time. Before comparing these results with other programs, it is necessary to verify that they use the same definition of throughput.

7.9 Reproducing the Measurements

The entire set of measurements can be reproduced from the `fdtd-hpc` directory using the following `make` commands. The first command builds the program with the recorded compiler options and runs the measurement script for eight grid widths and ten repetitions. The plots used in the analysis can then be regenerated from the same CSV file.

```
make bench
make bench_images
```

The `results/benchmark.csv` file begins with comments containing information about the compiler, processor, cache memory, and OpenMP environment, followed by the individual measurements in tabular form. The `accel` column identifies the kernel used, `threads` gives the number of threads, `width` is the grid width, `nsteps` is the number of time steps, `repeat` is the repetition index, and `seconds` is the measured execution time. The CSV header has the following form:

```
# cflags: -O3 -march=native -fopenmp -Wall -Iinclude
# compiler: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
# timestamp: 2026-08-15T15:13:59Z
# hostname: rtidev5
# cpu_model: 11th Gen Intel(R) Core(TM) i7-11700F @ 2.50GHz
# cores_physical: 8
# cores_logical: 16
# threads_per_core: 2
# numa_nodes: 1
# l2_cache: 4 MiB (8 instances)
# l3_cache: 16 MiB (1 instance)
# omp_dynamic: false
# omp_places: cores
# omp_proc_bind: close
accel,threads,width,nsteps,repeat,seconds
```

The `make bench_images` command processes the measurements from the CSV file and writes five plots whose names begin with `benchmark_` to `dist/results/images`. These plots show efficiency E_p, speedup S_p, and throughput P_p; speedup and throughput are also plotted as functions of the grid width K.

```
dist/results/images/
|-- benchmark_efficiency.png
|-- benchmark_speedup.png
|-- benchmark_speedup_k.png
|-- benchmark_throughput.png
`-- benchmark_throughput_k.png
```

Chapter 8

Results and Analysis

In the preceding chapter, we defined the simulation parameters and measurement procedure. In this chapter, we verify the resulting fields and analyze the performance of the serial and parallel implementations.

8.1 Electromagnetic Wave Propagation

The Gaussian source was placed at the center of the domain, where it creates two waves that propagate in opposite directions. The data after one hundred steps show E_z peaks of approximately 1.002 at cells 70 and 130, symmetrically positioned about the source. This symmetry, shown in Figure 8.1, confirms that the homogeneous interior region has identical properties to the left and right of the center.

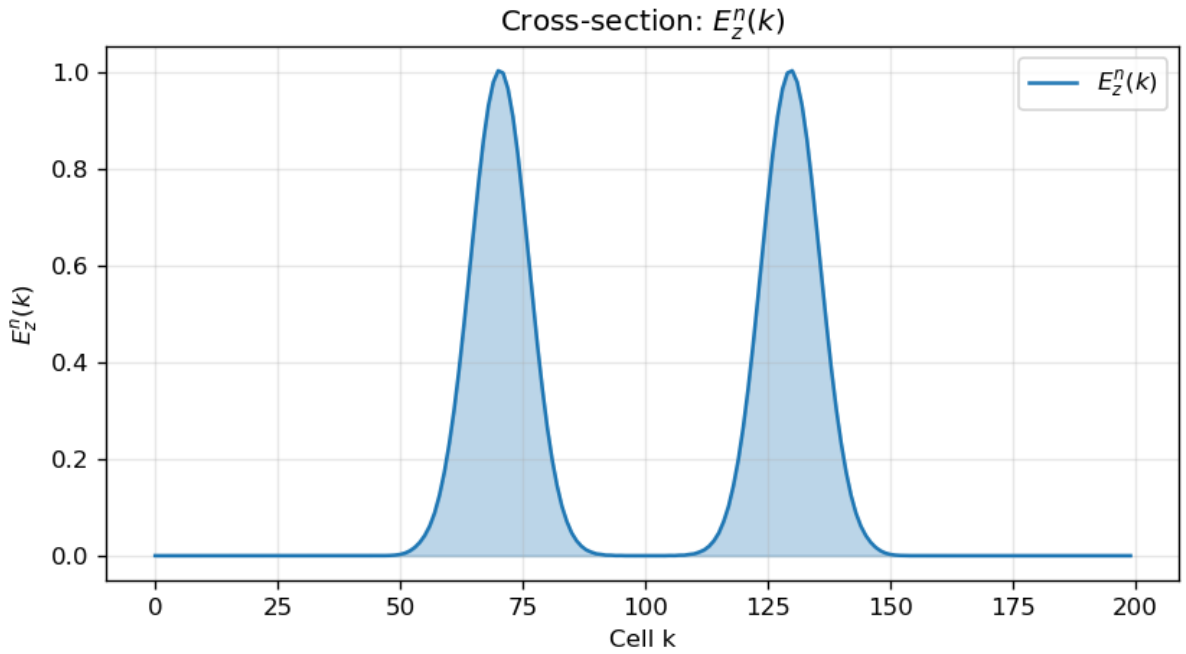


Figure 8.1: Electric field E_z after propagation of the Gaussian pulse

The magnetic component H_y follows the electric component, but the waves traveling in opposite directions have opposite signs. Its extrema in Figure 8.2 are approximately

$2.664 \cdot 10^{-3}$ at cell 70 and $-2.664 \cdot 10^{-3}$ at cell 129. The one-cell difference between the peak positions corresponds to the spatial staggering of the components on the Yee grid.

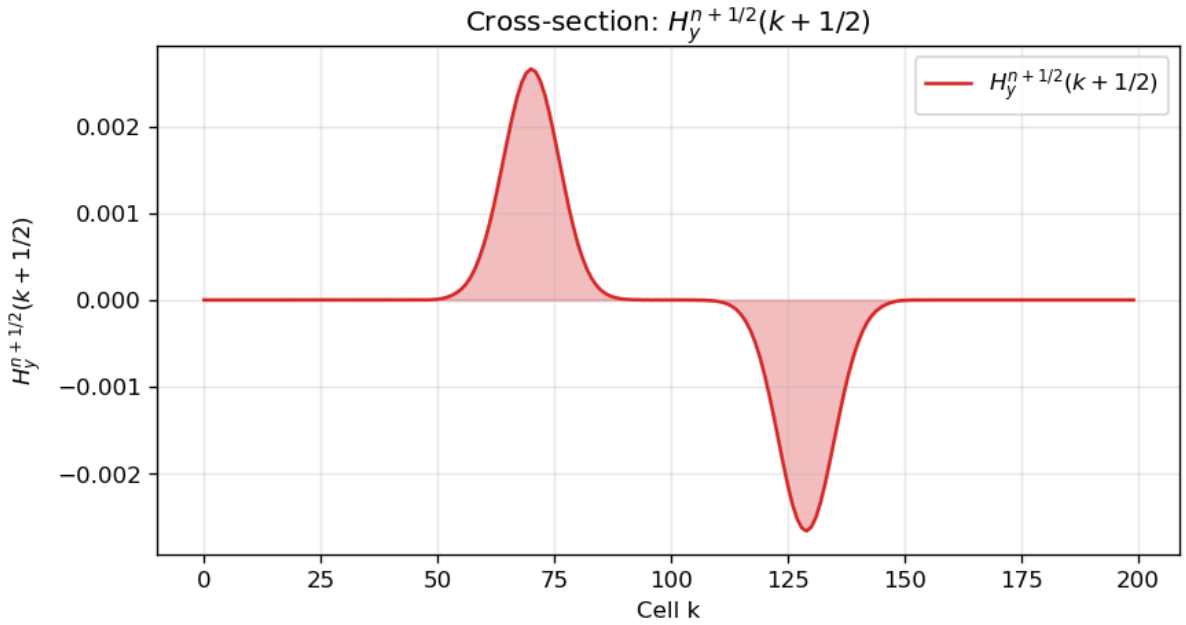


Figure 8.2: Magnetic field H_y after propagation of the Gaussian pulse

The three-dimensional illustration in Figure 8.3 shows the mutually perpendicular directions of the E_z and H_y components and the direction of plane-wave propagation along the x axis. It illustrates the splitting of the initial excitation and the movement of the two wave packets away from the center. The result corresponds to the expected solution of the one-dimensional wave equation in a homogeneous medium.

8.2 Analysis of PML Performance

The absorbing layer occupies twenty cells at both the left and right ends of the grid. Figure 8.4 identifies the interior region and the two parts of the layer, allowing the field position to be compared directly with the beginning of attenuation. The source is far from the boundary and does not affect the conductivity profile.

The σ_e and σ_m profiles are nonzero in the forty outermost cells, twenty on each side, and zero from cell 20 through cell 179. Electrical conductivity reaches 0.3667 S/m at the outer boundary, while the matched magnetic conductivity reaches 56,049. The gradual increase and staggered sampling of σ_m follow the position of H_y and reduce discrete reflection.

At the instant shown, the pulses are still within the useful region, so their shapes remain well preserved. In later steps, they enter the lossy profile and their amplitudes decrease before reaching the outer boundary. The absence of a strong returning pulse in the interior provides practical confirmation that the PML boundary is working correctly.

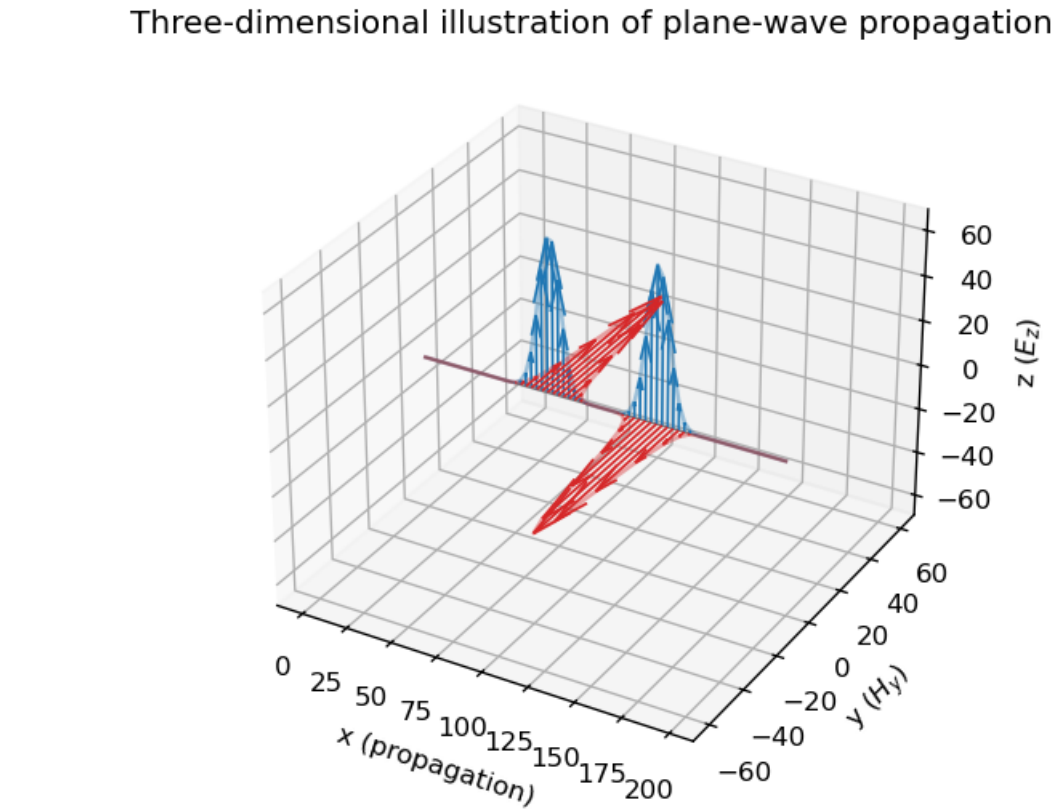


Figure 8.3: Three-dimensional illustration of plane-wave propagation

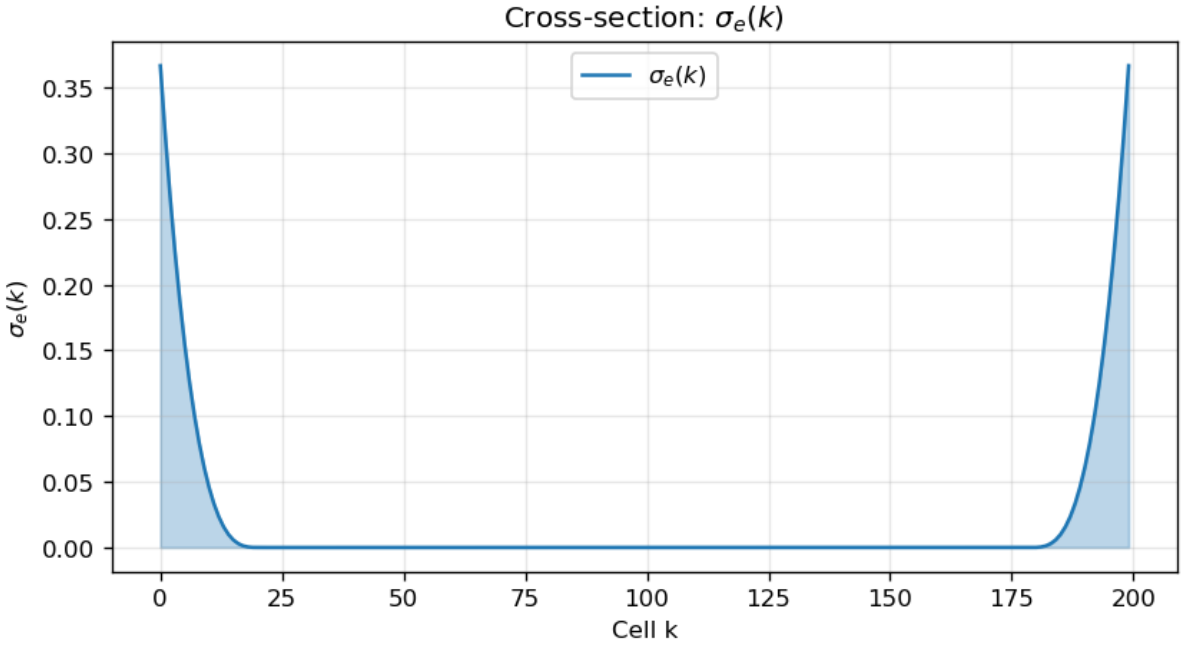


Figure 8.4: Positions of the left and right PML regions in the grid

8.3 Performance of the Serial Implementation

Serial throughput is not independent of grid width, even though every cell uses the same numerical expression. The median is 1,434.58 Mcells/s for $K = 10{,}000$, remains near 940 Mcells/s for widths from 40,000 through 160,000, and then falls to 518.22 Mcells/s for $K = 1{,}280{,}000$. The measured times and corresponding throughputs are given in Table 8.1.

Table 8.1: Median serial execution time and throughput

Width K	Time for 1,024 steps	Throughput
10,000	0.007138 s	1,434.58 Mcells/s
20,000	0.019227 s	1,065.17 Mcells/s
40,000	0.043378 s	944.27 Mcells/s
80,000	0.087035 s	941.23 Mcells/s
160,000	0.172724 s	948.57 Mcells/s
320,000	0.372540 s	879.58 Mcells/s
640,000	1.071730 s	611.50 Mcells/s
1,280,000	2.529292 s	518.22 Mcells/s

The decrease does not result from a greater number of operations per cell because the computational kernel does not change with grid width. The capacity and bandwidth of the memory hierarchy are possible causes, but hardware performance counters would be required to distinguish their effects. This conclusion is therefore inferred from the shape of the results and the array-access pattern.

The largest grid contains 128 times as many cells as the smallest, but its serial simulation takes approximately 354 times as long. Its throughput is approximately 2.77 times lower, showing that the cost of a single update grows with the working set. This serial behavior remains an important basis for interpreting the parallel results.

8.4 Speedup and Parallel Efficiency

Speedup increases up to a different thread count depending on the grid width. The largest recorded speedup is 6.218 for $K = 80{,}000$ with sixteen threads, while $K = 40{,}000$ reaches 6.109 with eight threads. For the largest grid, four threads provide a speedup of only 1.195 and sixteen threads provide 1.056, so additional threads do not overcome the limitation imposed by the large working set. Figure 8.5 shows this relationship, while Table 8.2 gives selected values.

Table 8.2: Median speedup for selected thread counts

K	1 thread	2 threads	4 threads	8 threads	16 threads
10,000	0.939	1.686	2.362	2.631	2.093
20,000	0.969	2.307	4.034	5.423	4.475
40,000	0.984	2.125	4.977	6.109	5.737
80,000	0.992	1.943	3.052	5.876	6.218
160,000	0.998	1.977	2.679	3.763	3.244

K	1 thread	2 threads	4 threads	8 threads	16 threads
320,000	0.999	1.973	2.902	3.203	2.796
640,000	1.003	1.555	1.842	1.816	1.676
1,280,000	0.999	1.135	1.195	1.130	1.056

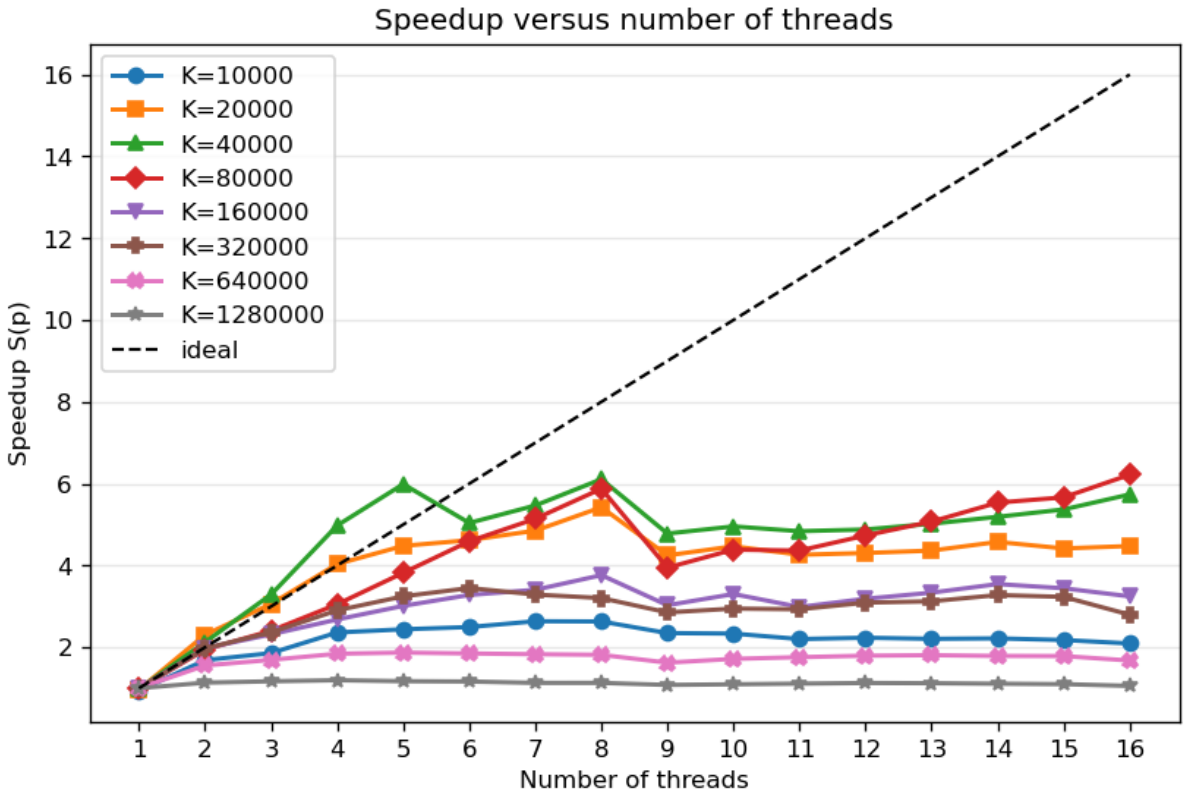


Figure 8.5: Speedup of the OpenMP implementation as a function of thread count

Efficiency on medium-sized grids sometimes exceeds one; for example, it reaches 1.244 for $K = 40{,}000$ with four threads, as shown in Figure 8.6. Such a superlinear result does not mean that the threads perform less arithmetic work. Rather, it is consistent with more favorable use of private caches, a change in processor frequency, or measurement variation. Without cache-miss counters, these effects cannot be isolated precisely.

Beyond eight threads, efficiency generally decreases because the second hardware threads on the physical cores become active. For $K = 80{,}000$, efficiency with sixteen threads is 0.389, whereas for $K = 1{,}280{,}000$ it is only 0.066. Speedup and efficiency should therefore be considered together instead of assuming in advance that the largest thread count is the best choice.

8.5 Effect of Thread Count

Parallel execution with one thread isolates the overhead of the OpenMP model from the benefit of additional cores. Its speedup is 0.939 for $K = 10{,}000$ and approaches one as the domain grows; for $K = 640{,}000$, the median is slightly above the serial reference.

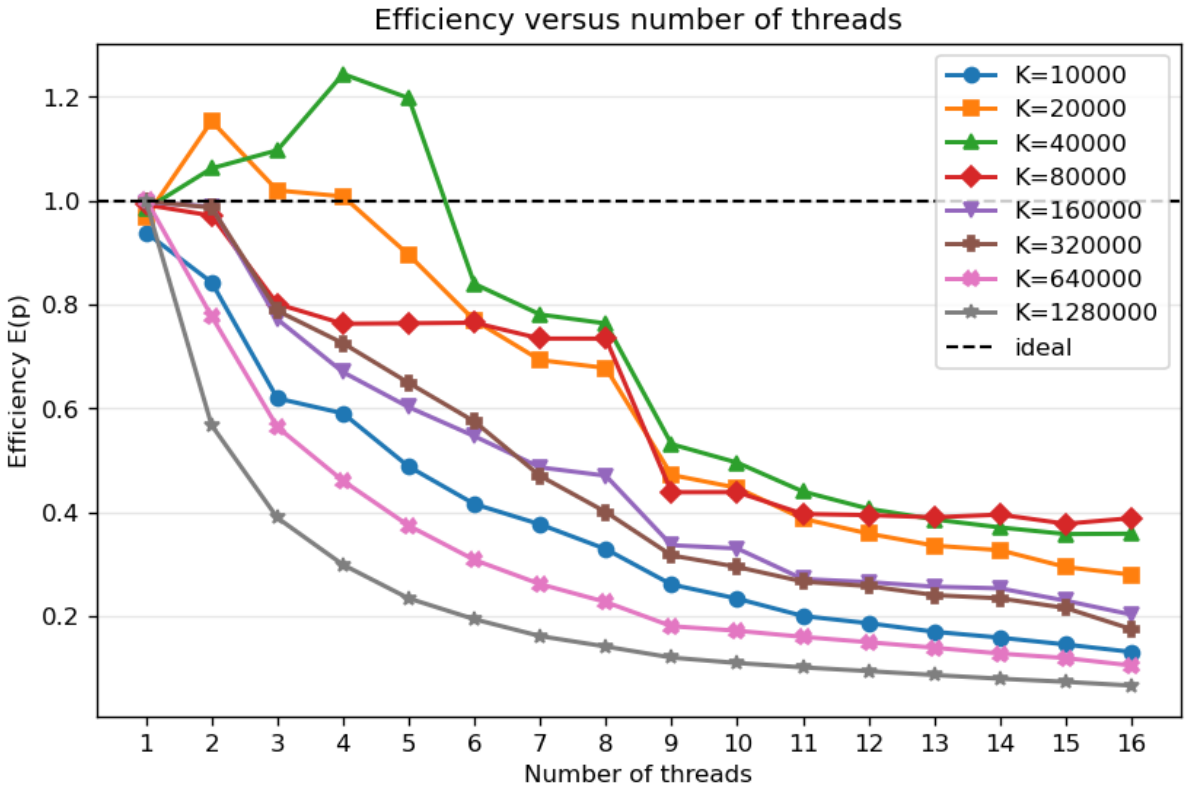


Figure 8.6: Parallel efficiency as a function of thread count

The cost of entering the parallel region and synchronizing is therefore visible primarily on smaller grids.

Figure 8.7 shows that throughput does not increase monotonically with thread count. Medium-sized grids reach approximately 5,700 to 5,850 Mcells/s, while the curves often decline after their maximum and show a transition between eight and nine threads. This transition coincides with the point at which physical cores begin to be shared, although thread placement and the memory subsystem may also affect the result.

The best measured thread counts are 7, 8, 8, 16, 8, 6, 5, and 4, respectively, as the grid width increases from 10,000 to 1,280,000. Differences between adjacent configurations are sometimes small, and the plots do not show intervals of variation, so these values should not be treated as fixed thresholds. Nevertheless, the clear decrease in the optimal thread count for large grids shows that automatic selection must take the domain size into account.

8.6 Effect of Simulation Domain Size

For $K = 1{,}280{,}000$, the selected configurations provide only 517.84 to 619.30 Mcells/s. Four threads perform best, with a speedup of 1.195, while adding threads beyond that point reduces throughput. This result contradicts the assumption that a larger domain always amortizes parallelization overhead more effectively.

Figure 8.8 shows throughput as a function of grid width. It increases up to medium grid sizes and then decreases for every selected thread count. With eight threads, the implementation achieves 5,776.34 Mcells/s for $K = 20{,}000$ and 5,768.27 Mcells/s for

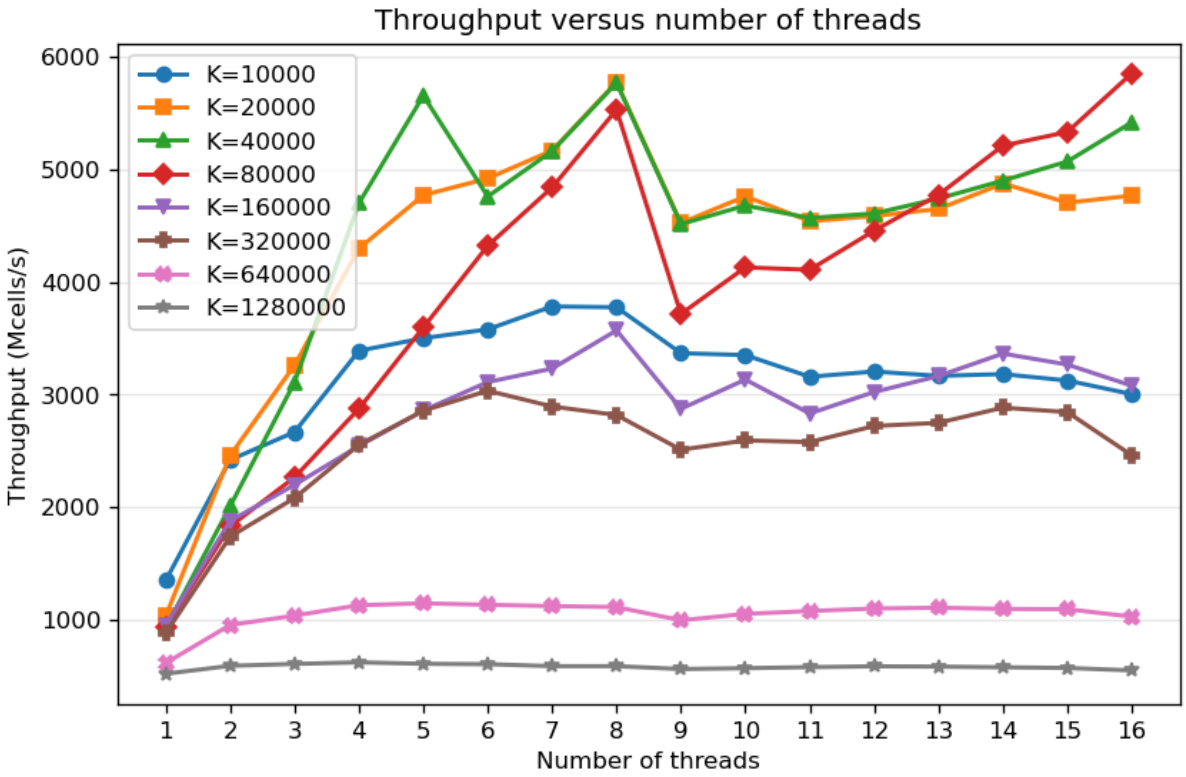


Figure 8.7: Parallel throughput for all grid widths and thread counts

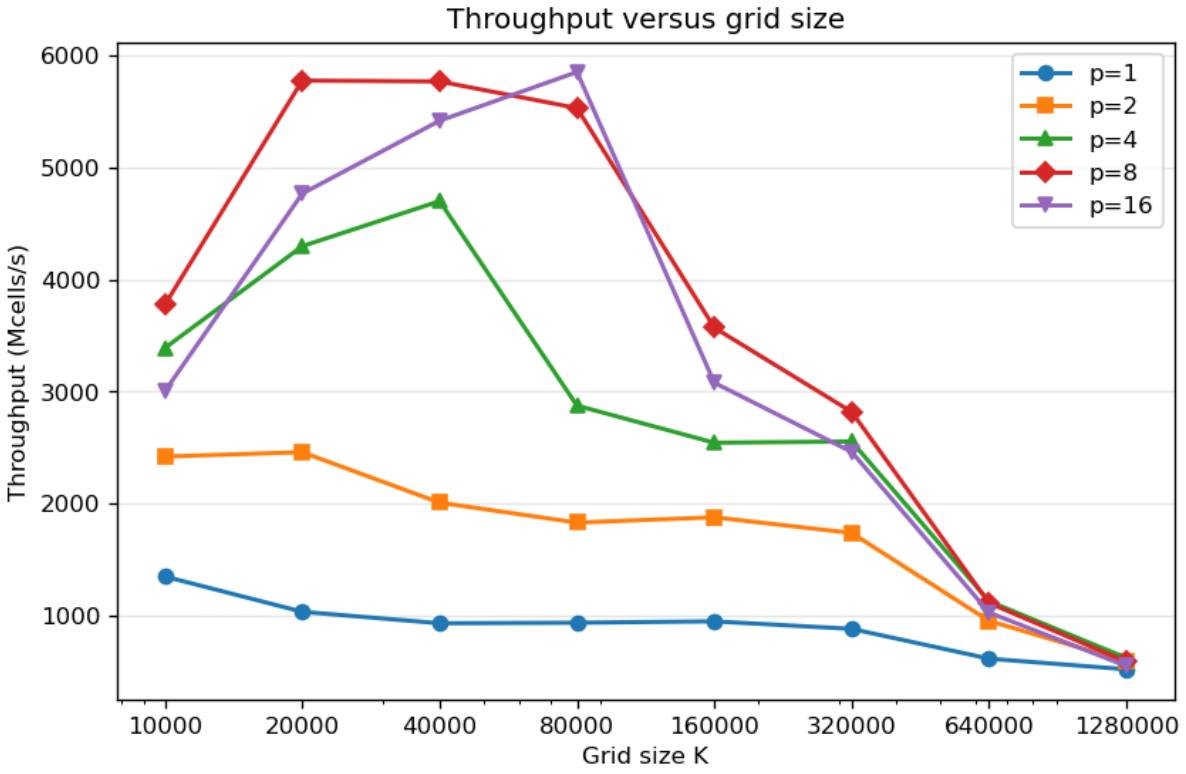


Figure 8.8: Throughput as a function of grid width

$K = 40{,}000$. The highest value, 5,852.47 Mcells/s, is achieved with sixteen threads for $K = 80{,}000$.

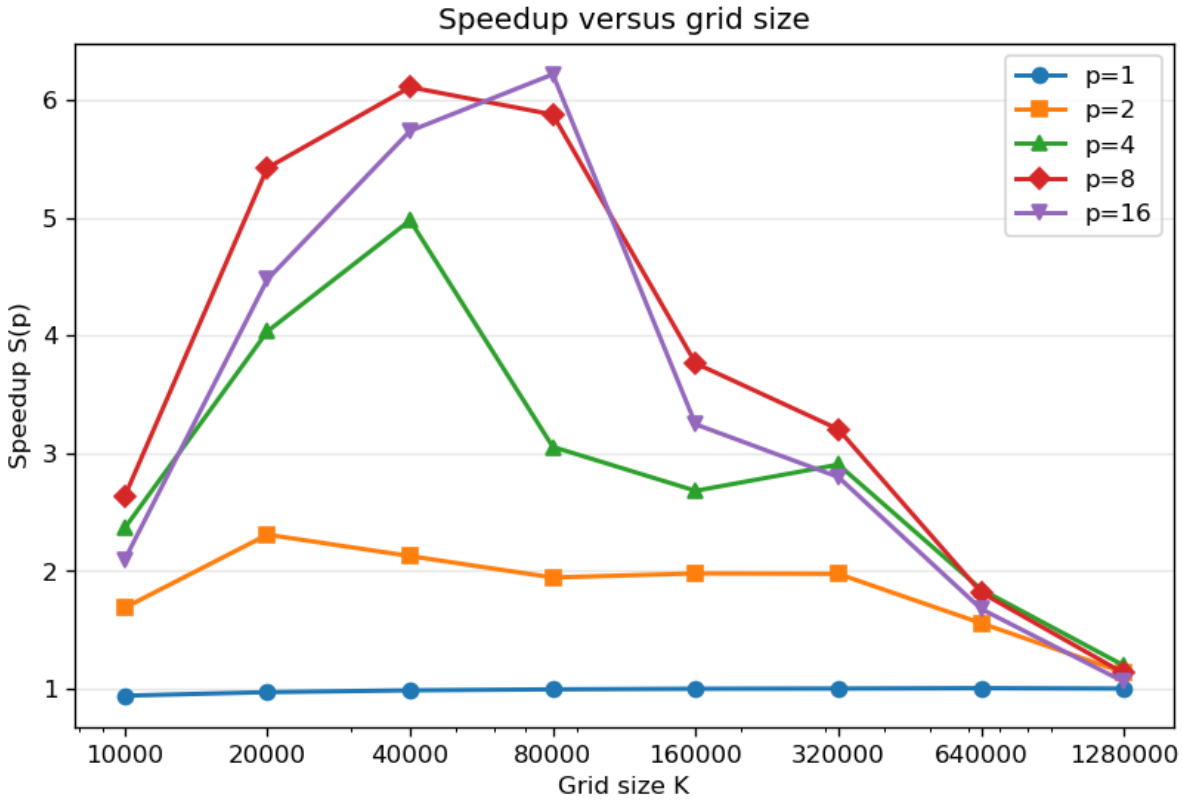


Figure 8.9: Speedup as a function of grid width

The speedup curves in Figure 8.9 have their highest values between 20,000 and 80,000 cells and then decline. With eight threads, speedup rises from 2.631 at 10,000 cells to 6.109 at 40,000 cells, after which it falls to 1.130 for the largest grid. This shape suggests a transition from thread-management overhead, through favorable cache utilization, to a limitation imposed by memory traffic.

8.7 Analysis of Performance Limitations

The results reveal three execution regimes. On the smallest grid, fixed OpenMP overhead is visible. Medium-sized grids make good use of multiple physical cores and their caches, while throughput decreases sharply on the largest grids. These changes in behavior explain why neither grid width nor thread count alone determines the best configuration.

With eight physical cores, ideal speedup would be eight before simultaneous multithreading becomes active, but the largest measured value in this range is 6.109. The remaining loss includes serial work, barriers, and the shared memory subsystem, while superlinear points at lower thread counts also indicate the influence of cache behavior. Without hardware performance counters, these causes can be presented only as reasoned hypotheses.

The implementation is most beneficial for medium-sized working sets that provide each thread with sufficient work while still using the cache hierarchy effectively. For very large domains, the thread count should be limited and actual throughput measured because

sixteen threads may be only slightly faster than serial execution. A future version could select the execution mode and thread count according to grid width using a short initial measurement.

Chapter 9

Conclusion

The preceding chapters presented the implementation in detail, together with an analysis of the serial and parallel implementations of the finite-difference time-domain method. This conclusion summarizes the results, limitations, and possible directions for further development.

9.1 Achieved Results

We derived a one-dimensional model for the E_z and H_y components with the spatial and temporal staggering of the Yee grid. Electrical and magnetic losses were incorporated into the discrete expressions, so the same form of the update equations applies both in the interior region and in the PML. The Gaussian-pulse plots show the expected splitting and propagation of the wave in both directions.

The model was implemented as a static library and a command-line program written in C. The state, fields, material, coefficients, and sources were separated into distinct structures, facilitating verification and further extension of the implementation. The serial and OpenMP kernels use the same data and identical update equations.

Measurements on eight physical and sixteen logical cores showed that the benefit of parallelization depends strongly on the size of the working set. The greatest speedup is 6.218 for 80,000 cells and sixteen threads, while 40,000 cells with eight threads reaches 6.109. On the largest grid of 1,280,000 cells, the best speedup is only 1.195 with four threads, demonstrating the limitation imposed by the memory subsystem.

9.2 Contributions of the Work

The theoretical part of this work connects Maxwell's equations, one-dimensional polarization, and the final expressions used in the program. Particular attention was given to the half-time-step position $H_y^{n+1/2}[k+1/2]$ and the spatial positions of the magnetic coefficients. This consistency simplifies verification of the loop bounds and the meaning of each program array.

The software portion demonstrates how the physical model can be separated from the execution method. A user program can change the material, source, or boundary without modifying the serial and parallel expressions. The same library supports simulation examples, resumption from a saved state, and reproducible performance measurements.

The experimental portion does not present only the best result, but compares multiple grid widths, thread counts, and repeated samples. This comparison reveals the transition from a regime dominated by thread-management overhead to one in which multiple cores provide a substantial benefit. The observed variation also warns that a single measurement may give a misleading picture.

9.3 Limitations of the Implemented Solution

The model is one-dimensional and cannot represent diffraction, complex geometry, or transverse modes. It considers only the polarization with the E_z and H_y components, while another polarization would require a separate set of functions. The results therefore cannot be transferred directly to every three-dimensional electromagnetic problem.

The PML implementation uses a fixed cubic profile and a fixed target reflection. The user can change the layer width, but cannot change its grading order or target reflection through the public interface. Absorption is verified from the shape of the fields, without a separate measurement of the reflection coefficient.

The measurements were performed on a single Intel Core i7-11700F processor with eight physical cores, so they do not characterize every possible environment. Hardware performance counters were not used to separate the effects of cache behavior, memory bandwidth, clock frequency, and execution units. Possible causes can therefore be inferred only from the code and the measured curves.

9.4 Directions for Future Development

A natural extension is the introduction of two- and three-dimensional domains with additional field components. A greater number of cells would increase the need for parallelism, but would also introduce more complex boundaries and data exchange between spatial blocks. The library's existing separation of components can serve as a foundation for new polarizations and simulation types.

The parallel kernel already retains the same team of threads throughout the entire sequence of time steps and distributes both spatial loops using `omp for` directives. Barriers between dependent phases and after source application nevertheless remain necessary in every step. Their cost could be reduced by more sophisticated spatial and temporal blocking, but such a change would require a different data organization and additional correctness verification.

Future measurements could include more processor architectures, higher core counts, and hardware performance counters. Automatic selection of serial or parallel execution could use the grid width and a short initial measurement. This would allow the library to retain a simple interface while choosing the more favorable execution method itself.

9.5 Recommendations for Use

Based on the medians of ten repetitions, Table 9.1 lists the fastest measured configuration for each grid width. Parallel execution provides a benefit in every tested case, but the most favorable thread count decreases for large working sets. These recommendations apply to the processor used, the thread-binding policy, and the current implementation.

Table 9.1: Fastest i7-11700F configurations

Grid width	Thread count	Median speedup	Median throughput
10,000	7	2.636	3,782.09 Mcells/s
20,000	8	5.423	5,776.34 Mcells/s
40,000	8	6.109	5,768.27 Mcells/s
80,000	16	6.218	5,852.47 Mcells/s
160,000	8	3.763	3,568.99 Mcells/s
320,000	6	3.448	3,033.04 Mcells/s
640,000	5	1.870	1,143.72 Mcells/s
1,280,000	4	1.195	619.30 Mcells/s

Medium-sized grids of 20,000 to 80,000 cells make the best use of the available cores and reduce execution time by factors ranging from 5.423 to 6.218. As the grid grows further, the optimal thread count decreases, and on a grid of 1,280,000 cells four threads provide a benefit of only 19.5%. These recommendations should be reevaluated after changing the processor, compiler, thread binding, or build configuration.

The choice should not be based solely on the number of available threads, but also on the grid width, timing stability, and overall system load. When multiple independent simulations are executed, it may be preferable to assign them to separate groups of cores instead of allocating all sixteen threads to each one. For a single simulation, the measured maximum throughput should be selected because the largest thread count is not automatically the fastest.

References

[1] A. R. Đorđević, *Elektromagnetika*. Belgrade: Akademska misao, 2012.

[2] D. M. Sullivan, *Electromagnetic Simulation Using the FDTD Method*, 2nd ed. Hoboken, NJ: Wiley-IEEE Press, 2013.

[3] K. S. Yee, "Numerical Solution of Initial Boundary Value Problems Involving Maxwell's Equations in Isotropic Media," *IEEE Transactions on Antennas and Propagation*, vol. 14, no. 3, pp. 302–307, 1966.

[4] A. Taflove and S. C. Hagness, *Computational Electrodynamics: The Finite-Difference Time-Domain Method*, 3rd ed. Artech House, 2005.

[5] J.-P. Bérenger, "A Perfectly Matched Layer for the Absorption of Electromagnetic Waves," *Journal of Computational Physics*, vol. 114, no. 2, pp. 185–200, 1994.

[6] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*. 2021.

[7] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in *AFIPS conference proceedings*, 1967, pp. 483–485.

List of Abbreviations

ABC Absorbing Boundary Condition
API Application Programming Interface
CSV Comma-Separated Values
FDTD Finite-Difference Time-Domain
HPC High-Performance Computing
OpenMP Open Multi-Processing
PEC Perfect Electric Conductor
PML Perfectly Matched Layer

List of Figures

3.1 Positions of the E_z and H_y components on a one-dimensional Yee grid . 12
4.1 Electrical conductivity in the left and right PML layers 17
6.1 Thread fork and join in the OpenMP fork-join model 27
8.1 Electric field E_z after propagation of the Gaussian pulse 36
8.2 Magnetic field H_y after propagation of the Gaussian pulse 37
8.3 Three-dimensional illustration of plane-wave propagation 38
8.4 Positions of the left and right PML regions in the grid 38
8.5 Speedup of the OpenMP implementation as a function of thread count . 40
8.6 Parallel efficiency as a function of thread count 41
8.7 Parallel throughput for all grid widths and thread counts 42
8.8 Throughput as a function of grid width 42
8.9 Speedup as a function of grid width 43
E.1 Electric field E_z produced by the simulation plotting script 72
E.2 Speedup of the OpenMP implementation produced by the benchmark plotting script 73

List of Tables

2.1 Material quantities in the theoretical model 10
5.1 Main components of the software solution 19
6.1 Data accesses in one iteration 30
7.1 Measurement environment recorded with the results 31
7.2 Basic measurement parameters 32
8.1 Median serial execution time and throughput 39
8.2 Median speedup for selected thread counts 39
9.1 Fastest i7-11700F configurations 47
A.1 Fields and material properties 52

Appendix A

Notation and Symbols

This appendix provides Table A.1, which lists the most important physical and numerical quantities introduced in the preceding chapters. Quantities marked with an arrow are vectors and depend on three spatial coordinates and time.

The discrete indices k and n denote a spatial point and a time step. The electric field is written as $E_z^n[k]$, while the magnetic field is staggered and has the form $H_y^{n+1/2}[k+1/2]$. The increments Δx and Δt are related by the Courant number S.

Table A.1: Fields and material properties

Symbol	Meaning
$\vec{E}$, $E_z(x,t)$	electric-field vector and component (V/m)
$\vec{H}$, $H_y(x,t)$	magnetic-field vector and component (A/m)
$\vec{D}$	electric displacement vector (C/m^2)
$\vec{B}$	magnetic flux density vector (T)
$\varepsilon(x,t)$, $\varepsilon_r(x,t)$	absolute and relative permittivity (F/m, 1)
$\mu(x,t)$, $\mu_r(x,t)$	absolute and relative permeability (H/m, 1)
$\sigma_e(x,t)$, $\sigma_m(x,t)$	electric and magnetic conductivity (S/m, adjusted unit)
K	number of spatial cells
N	number of time steps
$C_{e1}[k]$, $C_{e2}[k]$	electric-field update coefficients
$C_{h1}[k+1/2]$, $C_{h2}[k+1/2]$	magnetic-field update coefficients
p	number of processing threads
T_p	execution time with p threads
S_p	parallel speedup
E_p	parallel efficiency

Appendix B

Derivation of the One-Dimensional Model

Chapter 2 presented the final system of differential equations for the H_y and E_z components. In this appendix, we derive that system in detail, starting from the differential forms of Faraday's law in Equation (2.3) and the Ampère-Maxwell law in Equation (2.4).

$$\nabla \times \vec{E} = -\frac{\partial \vec{B}}{\partial t} \tag{B.1}$$

$$\nabla \times \vec{H} = \vec{J} + \frac{\partial \vec{D}}{\partial t} \tag{B.2}$$

B.1 The Nabla Operator

The nabla operator ∇ provides a compact notation for spatial derivatives of a vector field. In the Cartesian coordinate system, its components are partial-derivative operators with respect to the x, y, and z coordinates, while $\vec{i}_x$, $\vec{i}_y$, and $\vec{i}_z$ denote the corresponding unit vectors.

$$\nabla = \vec{i}_x \frac{\partial}{\partial x} + \vec{i}_y \frac{\partial}{\partial y} + \vec{i}_z \frac{\partial}{\partial z} \tag{B.3}$$

The curl of a vector field is obtained from the cross product of the nabla operator and that field. For the electric field $\vec{E} = (E_x, E_y, E_z)$, this product can be written as a determinant whose first row contains the unit vectors, whose second row contains the spatial derivatives, and whose third row contains the field components.

$$\nabla \times \vec{E} = \begin{vmatrix} \vec{i}_x & \vec{i}_y & \vec{i}_z \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ E_x & E_y & E_z \end{vmatrix} \tag{B.4}$$

The same procedure applies to the magnetic field $\vec{H} = (H_x, H_y, H_z)$. The following sections expand the curls of the electric and magnetic fields because they determine the spatial coupling of the components in the time-evolution equations.

B.2 Time-Domain Form of the Equations

We begin with the two curl equations because they contain the time derivatives of the electric and magnetic flux densities. Into them we substitute the constitutive relations $\vec{D} = \varepsilon\vec{E}$, $\vec{B} = \mu\vec{H}$, and $\vec{J} = \sigma_e\vec{E}$ from Equations (2.5), (2.6), and (2.7).

In Faraday's law, we isolate the time derivative of the magnetic field; in the Ampère-Maxwell law, we isolate the time derivative of the electric field. The parameters μ, ε, and σ_e are treated as time-invariant properties of a linear, isotropic medium.

$$\frac{\partial\vec{H}}{\partial t} = -\frac{1}{\mu}\nabla\times\vec{E} \tag{B.5}$$

$$\frac{\partial\vec{E}}{\partial t} = \frac{1}{\varepsilon}\nabla\times\vec{H} - \frac{\sigma_e}{\varepsilon}\vec{E} \tag{B.6}$$

In the resulting equations, the time derivative of the magnetic field depends on the curl of the electric field, while the time derivative of the electric field depends on the curl of the magnetic field and on electrical losses. The curls still contain all three spatial components, so the next section expands them to obtain a scalar equation for each field component.

B.3 Expansion of the Curls and Scalar Form of the Equations

We expand the curl of the electric field in terms of the E_x, E_y, and E_z components and the curl of the magnetic field in the same manner using H_x, H_y, and H_z. Each curl component contains the difference of two spatial derivatives that describe the variation of the field in the plane normal to the direction under consideration.

$$\nabla\times\vec{E} = \left(\frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z}, \frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x}, \frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y}\right) \tag{B.7}$$

$$\nabla\times\vec{H} = \left(\frac{\partial H_z}{\partial y} - \frac{\partial H_y}{\partial z}, \frac{\partial H_x}{\partial z} - \frac{\partial H_z}{\partial x}, \frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y}\right) \tag{B.8}$$

The expanded curls are substituted into their corresponding time-domain equations. The first three scalar equations describe the time variation of the magnetic-field components, while the next three describe the time variation of the electric-field components in the presence of electrical losses.

$$\frac{\partial H_x}{\partial t} = -\frac{1}{\mu}\left(\frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z}\right) \tag{B.9}$$

$$\frac{\partial H_y}{\partial t} = -\frac{1}{\mu}\left(\frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x}\right) \tag{B.10}$$

$$\frac{\partial H_z}{\partial t} = -\frac{1}{\mu}\left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y}\right) \tag{B.11}$$

$$\frac{\partial E_x}{\partial t} = \frac{1}{\varepsilon}\left(\frac{\partial H_z}{\partial y} - \frac{\partial H_y}{\partial z}\right) - \frac{\sigma_e}{\varepsilon}E_x \tag{B.12}$$

$$\frac{\partial E_y}{\partial t} = \frac{1}{\varepsilon}\left(\frac{\partial H_x}{\partial z} - \frac{\partial H_z}{\partial x}\right) - \frac{\sigma_e}{\varepsilon}E_y \tag{B.13}$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon}\left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y}\right) - \frac{\sigma_e}{\varepsilon}E_z \tag{B.14}$$

These six equations describe the time variation of all electromagnetic-field components in three-dimensional space. For the model in this work, we retain only the spatial and vector components required for propagation along a single axis.

B.4 One-Dimensional System of Equations

We consider a plane electromagnetic wave propagating in one direction. Without loss of generality, propagation is chosen along the x axis, so the fields depend only on the coordinate x and time t. Derivatives with respect to the transverse coordinates y and z are therefore zero.

For the polarization, we choose the electric field in the z direction and the magnetic field in the y direction. Only the E_z and H_y components are retained, while the remaining four components are set to zero.

$$\frac{\partial}{\partial y} = 0, \qquad \frac{\partial}{\partial z} = 0, \qquad E_x = 0, \qquad E_y = 0, \qquad H_x = 0, \qquad H_z = 0 \tag{B.15}$$

We first apply the conditions of the one-dimensional model to the magnetic-field equations. The equations for H_x and H_z become trivial, while the equation for H_y retains the spatial derivative of E_z with respect to x.

$$\frac{\partial H_y}{\partial t} = \frac{1}{\mu}\frac{\partial E_z}{\partial x} \tag{B.16}$$

We then apply the same conditions to the electric-field equations. The equations for E_x and E_y become trivial, while the equation for E_z retains the spatial derivative of H_y and the electrical-loss term.

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon}\frac{\partial H_y}{\partial x} - \frac{\sigma_e}{\varepsilon}E_z \tag{B.17}$$

The remaining two equations form a coupled one-dimensional system that is first order in space and time. A change in E_z determines the new value of H_y, while a change in H_y in turn determines the new value of E_z.

B.5 Magnetic Currents and a Symmetric System of Equations

Faraday's law and the Ampère-Maxwell law have similar structures, but Faraday's law lacks a term corresponding to the free-charge current density $\vec{J}$. This asymmetry results from the physical absence of magnetic monopoles that would act as carriers of magnetic charge.

For the purposes of a symmetric mathematical model, however, we can formally introduce the magnetic current density $\vec{J}_m$ and denote the existing quantity $\vec{J}$ as the electric current density $\vec{J}_e$. In the linear, isotropic medium under consideration, both current densities are related to their corresponding fields through constitutive relations, where σ_m denotes the fictitious magnetic conductivity and σ_e the electrical conductivity.

$$\vec{J}_m = \sigma_m \vec{H} \tag{B.18}$$

$$\vec{J}_e = \sigma_e \vec{E} \tag{B.19}$$

Using the introduced magnetic current density, we extend Faraday's law with the term $-\vec{J}_m$, while the Ampère-Maxwell law retains the same form with the notation $\vec{J}_e$. This gives a symmetric system of vector equations.

$$\nabla \times \vec{E} = -\vec{J}_m - \frac{\partial \vec{B}}{\partial t} \tag{B.20}$$

$$\nabla \times \vec{H} = +\vec{J}_e + \frac{\partial \vec{D}}{\partial t} \tag{B.21}$$

Following the same procedure as in the preceding sections, we substitute the constitutive relations and apply the selected propagation direction and polarization. This yields the symmetric one-dimensional system for the H_y and E_z components.

$$\frac{\partial H_y}{\partial t} = \frac{1}{\mu}\frac{\partial E_z}{\partial x} - \frac{\sigma_m}{\mu} H_y \tag{B.22}$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon}\frac{\partial H_y}{\partial x} - \frac{\sigma_e}{\varepsilon} E_z \tag{B.23}$$

The resulting system consists precisely of Equations (2.16) and (2.17), which Chapter 2 uses as the starting point for discretization. Introducing fictitious magnetic conductivity makes it possible to match electric and magnetic losses, which is important for implementing absorbing boundary conditions such as the Berenger PML presented in Chapter 4.

Appendix C

Pseudocode for the Finite-Difference Method Implementation

This appendix presents the FDTD algorithm as simplified C code. The blocks follow the order in which parameters are prepared, fields and materials are configured, coefficients are calculated, and time stepping is performed.

A more detailed analysis of the algorithm can be found in Sullivan's *Electromagnetic Simulation Using the FDTD Method* [2] and Taflove and Hagness's *Computational Electrodynamics: The Finite-Difference Time-Domain Method* [4].

C.1 Constants and Parameters

The constants `K` and `N` specify the number of spatial cells and time steps. The constants `MU_0`, `EPS_0`, and `C_0` describe free space and the wave speed from Equations (2.12) and (2.18), while `PI` is used to calculate the sinusoidal excitation from Equation (3.19).

```
// Constants
#define K 200
#define N 100
#define PI 3.14159265
#define MU_0 1.25663706e-6
#define EPS_0 8.85418782e-12
#define C_0 299792458.0
```

C.2 Arrays

The `Hy` and `Ez` arrays represent the field components from Equation (2.10), while the `mu`, `eps`, `sigma_m`, and `sigma_e` arrays describe the material parameters of the one-dimensional domain. The `Ch1`, `Ch2`, `Ce1`, and `Ce2` coefficients correspond to Equations (3.14)–(3.17) and are used directly when updating the fields. The quantities `S`, `dx`, and `dt` are related by the Courant condition in Equations (3.21) and (3.22).

```
// Fields
double Hy[K], Ez[K];

// Material
```

```
double mu[K], eps[K], sigma_m[K], sigma_e[K];

// Update coefficients
double Ch1[K], Ch2[K], Ce1[K], Ce2[K];

// Simulation parameters
double S, dx, dt;
```

C.3 Function Declarations

The declarations introduce the operations used below and show the organization of the pseudocode. The `Init()`, `SetHy()`, `SetEz()`, `SetMu()`, `SetEps()`, `SetPEC()`, `SetPML()`, and `UpdateCoeffs()` functions prepare the simulation, while `UpdateHy()`, `UpdateEz()`, `AddPulseEz()`, and `AddSineEz()` each perform one part of a time step. The `Fdtd_Run()` function defines their order, and `main()` provides an example of a complete run.

```
// Initialization
void Init();

// Set fields and materials
void SetHy(double value, size_t start, size_t end);
void SetEz(double value, size_t start, size_t end);
void SetMu(double value, size_t start, size_t end);
void SetEps(double value, size_t start, size_t end);

// Configure boundaries
void SetPEC();
void SetPML(size_t pml_width);

// Update coefficients
void UpdateCoeffs();

// Update fields and excitations
void UpdateHy();
void UpdateEz();
void AddPulseEz(int n, double n0, double spread, size_t kc);
void AddSineEz(int n, double amp, double freq, double phase, size_t kc);

// Main functions
void Fdtd_Run(int nsteps, bool use_sine);
int main();
```

C.4 Simulation Initialization

The `Init()` function sets the Courant number to its default value, $S = 0.5$. It then calculates `dt` from the spatial increment `dx` and the speed `C_0` according to Equation (3.22). The resulting time step is later used by `UpdateCoeffs()` to calculate the update coefficients.

```
// Initialize the time step
void Init()
{
    S = 0.5;
    dt = S * dx / C_0;
}
```

C.5 Setting Fields and Materials

The `SetHy()` and `SetEz()` functions set the magnetic- and electric-field values in the selected half-open range `[start, end)`. Calling these functions with a value of zero over the entire domain produces the initial state.

```
// Magnetic field
void SetHy(double value, size_t start, size_t end)
{
    for (size_t k = start; k < end; k++) {
        Hy[k] = value;
    }
}

// Electric field
void SetEz(double value, size_t start, size_t end)
{
    for (size_t k = start; k < end; k++) {
        Ez[k] = value;
    }
}
```

The `SetMu()` and `SetEps()` functions assign permeability and permittivity to the selected part of the domain. These arrays determine wave speed and impedance according to Equations (2.11), (2.12), and (2.18). `UpdateCoeffs()` is called after each change because the update coefficients depend directly on the material parameters.

```
// Magnetic permeability
void SetMu(double value, size_t start, size_t end)
{
    for (size_t k = start; k < end; k++) {
        mu[k] = value;
    }

    // Update coefficients
    UpdateCoeffs();
}

// Electric permittivity
void SetEps(double value, size_t start, size_t end)
{
```

```
    for (size_t k = start; k < end; k++) {
        eps[k] = value;
    }

    // Update coefficients
    UpdateCoeffs();
}
```

C.6 Configuring the Boundary Layer

The `SetPEC()` function sets the electric and magnetic conductivities to zero throughout the domain. This removes the loss terms from the differential Equations (2.16) and (2.17), after which a call to `UpdateCoeffs()` recalculates the coefficients. The `UpdateHy()` and `UpdateEz()` functions consequently retain the same form for differently configured boundaries.

```
// PEC boundary condition
void SetPEC()
{
    for (size_t k = 0; k < K; k++) {
        // PEC conductivities
        sigma_m[k] = 0.0;
        sigma_e[k] = 0.0;
    }

    // Update coefficients
    UpdateCoeffs();
}
```

From the layer width, target reflection, and local impedance, the `SetPML()` function determines the maximum conductivity according to Equations (4.2) and (4.3). Electric and magnetic points have separate normalized depths, `de` and `dh`, which produce the polynomial conductivity profiles from Equations (4.4) and (4.5). After the `sigma_m` and `sigma_e` arrays have been set, `UpdateCoeffs()` incorporates their effects into the update coefficients.

```
// PML boundary condition
void SetPML(size_t pml_width)
{
    // PML parameters
    int order = 3;
    double refl = 1.0e-6;
    double width = pml_width;
    double thickness = width * dx;

    // Conductivity grading
    for (size_t k = 0; k < K; k++) {
        // Maximum PML conductivity
```

```
        double eta = sqrt(mu[k] / eps[k]);
        double sigma_max = -(order + 1) * log(refl)
                         / (2.0 * eta * thickness);

        // PML depth of the E point
        double xe = k;
        double de = 0.0;
        if (xe < width) {
            de = (width - xe) / width;
        } else if (xe >= K - width) {
            de = (xe - (K - width) + 1.0) / width;
        }

        // PML depth of the H point
        double xh = k + 0.5;
        double dh = 0.0;
        if (xh < width) {
            dh = (width - xh) / width;
        } else if (xh >= K - width) {
            dh = (xh - (K - width) + 1.0) / width;
        }

        // PML conductivities and losses
        sigma_m[k] = sigma_max * pow(dh, order) * mu[k] / eps[k];
        sigma_e[k] = sigma_max * pow(de, order);
    }

    // Update coefficients
    UpdateCoeffs();
}
```

C.7 Calculating Update Coefficients

For every cell, the `UpdateCoeffs()` function first calculates the dimensionless loss factors `a` and `b` from Equation (3.13). The `Ce1`, `Ce2`, `Ch1`, and `Ch2` coefficients are then obtained according to Equations (3.14)–(3.17). The coefficients are prepared before the time loop, allowing `UpdateHy()` and `UpdateEz()` to use them without repeating division in every step.

```
// Update coefficients from material properties
void UpdateCoeffs()
{
    for (size_t k = 0; k < K; k++) {
        // Semi-implicit loss terms
        double b = dt * sigma_m[k] / (2.0 * mu[k]);
        double a = dt * sigma_e[k] / (2.0 * eps[k]);

        // Hy coefficients
```

```
        Ch1[k] = (1.0 - b) / (1.0 + b);
        Ch2[k] = dt / (dx * mu[k] * (1.0 + b));

        // Ez coefficients
        Ce1[k] = (1.0 - a) / (1.0 + a);
        Ce2[k] = dt / (dx * eps[k] * (1.0 + a));
    }
}
```

C.8 Updating Fields and Excitations

The `UpdateHy()` function calculates the new magnetic field from the current electric field for indices from zero through `K - 2`, according to the update Equation (3.11). Only after it has finished do we call `UpdateEz()`, which uses the updated magnetic field and processes electric points from index one through `K - 1` according to Equation (3.12). The loop bounds correspond to the spatial staggering of E_z and H_y on the Yee grid in Equation (3.2).

```
// Update Hy by one step
void UpdateHy()
{
    for (size_t k = 0; k < K - 1; k++) {
        Hy[k] = Ch1[k] * Hy[k] + Ch2[k] * (Ez[k + 1] - Ez[k]);
    }
}

// Update Ez by one step
void UpdateEz()
{
    for (size_t k = 1; k < K; k++) {
        Ez[k] = Ce1[k] * Ez[k] + Ce2[k] * (Hy[k] - Hy[k - 1]);
    }
}
```

The `AddPulseEz()` function calculates the Gaussian pulse from Equation (3.18), while `AddSineEz()` creates a sinusoidal excitation according to Equation (3.19). Both functions add the resulting value to cell `kc`, implementing the soft source injection from Equation (3.20). The source therefore does not replace the value previously calculated by `UpdateEz()`.

```
// Gaussian excitation
void AddPulseEz(int n, double n0, double spread, size_t kc)
{
    double pulse = exp(-0.5 * pow((n0 - (double) n) / spread, 2.0));
    Ez[kc] += pulse;
}

// Sinusoidal excitation
```

```
void AddSineEz(int n, double amp, double freq, double phase, size_t kc)
{
    double sine = amp * sin(2.0 * PI * freq * (double) n + phase);
    Ez[kc] += sine;
}
```

C.9 Main Function

The `Fdtd_Run()` function performs the specified number of time steps and selects the source type according to the `use_sine` parameter. In every step, it first calls `UpdateHy()`, then `UpdateEz()`, and finally `AddSineEz()` or `AddPulseEz()`. This order directly follows the update Equations (3.11) and (3.12), after which the source is added according to Equation (3.20).

```
// FDTD algorithm
void Fdtd_Run(int nsteps, bool use_sine)
{
    for (int n = 0; n < nsteps; n++) {
        // Update Hy
        UpdateHy();

        // Update Ez
        UpdateEz();

        // Inject the source
        if (use_sine) {
            AddSineEz(n, 1.0, 0.05, 0.0, K / 2);
        } else {
            AddPulseEz(n, 40.0, 12.0, K / 2);
        }
    }
}
```

The `main()` function prepares an example simulation before entering the time loop. Calls to `SetHy()` and `SetEz()` establish zero initial field values, while `SetMu()` and `SetEps()` configure free space according to Equation (2.18). `SetPML()` then configures the absorbing layers, and `Fdtd_Run()` performs `N` steps with a Gaussian source because the `use_sine` parameter is `false`.

```
// Main function
int main()
{
    // Prepare the simulation
    Init();

    // Set fields and materials
    SetHy(0.0, 0, K);
    SetEz(0.0, 0, K);
```

```
    SetMu(MU_0, 0, K);
    SetEps(EPS_0, 0, K);

    // Configure boundary conditions
    SetPML(20);

    // Simulate N steps with a Gaussian source
    Fdtd_Run(N, false);
}
```

Appendix D

Project Architecture and Organization

This appendix presents the organization of the source project, the built distribution, and the experimental results. For each part, we first describe the role of its directories and then provide a tree with a brief description of each file.

D.1 Project Directory

The root `fdtd-hpc` directory contains the library source code, two user-facing programs, documentation, and utility scripts. Its organization separates the public programming interface from the implementation and from the procedures used to produce results.

```
fdtd-hpc/
|-- include/                        Public header directory
|   |-- fdtd/                       FDTD library headers
|   |   |-- Ez/                     Ez simulation headers
|   |   |   |-- fdtd_ez_coeffs.h    Update-coefficient API
|   |   |   |-- fdtd_ez_fields.h    Field API
|   |   |   |-- fdtd_ez_kernels.h   Serial and OpenMP kernel API
|   |   |   |-- fdtd_ez_material.h Material API
|   |   |   |-- fdtd_ez_sim.h       Simulation API
|   |   |   |-- fdtd_ez_sources.h   Source API
|   |   |   `-- fdtd_ez_state.h     State API
|   |   |-- fdtd_accel.h            Acceleration modes
|   |   |-- fdtd_alloc.h            Memory allocation
|   |   |-- fdtd_const.h            Physical constants
|   |   |-- fdtd_ez.h               Ez module
|   |   `-- fdtd_types.h            Data types
|   `-- fdtd.h                       Public API entry point
|-- docs/                            English document sources and results
|-- src/                             Source-file directory
|   |-- fdtd/                        FDTD library implementation
|   |   |-- Ez/                      Ez simulation implementation
|   |   |   |-- fdtd_ez_coeffs.c    Coefficient calculation
|   |   |   |-- fdtd_ez_fields.c    Field management
```

```
|   |   |   |-- fdtd_ez_kernels.c  Serial and OpenMP kernels
|   |   |   |-- fdtd_ez_material.c Material management
|   |   |   |-- fdtd_ez_sim.c      Simulation management
|   |   |   |-- fdtd_ez_sources.c  Source management
|   |   |   `-- fdtd_ez_state.c    State management
|   |   `-- fdtd_alloc.c           Aligned memory allocation
|   |-- benchmark.c                Performance measurement program
|   `-- main.c                     Simulation program
|-- tools/                          Utility scripts
|   |-- benchmark.sh               Repeated performance measurements
|   |-- get_sys_info.sh            System-information collection
|   |-- plot_benchmark.py          Performance plots
|   |-- plot_sim.py                Simulation plots
|   `-- plot_figures.py            Standalone illustrations
|-- Makefile                       Build and experiment rules
`-- README.md                      Project description
```

The `include` directory contains the library's public declarations. The `fdtd.h` header is the interface entry point, while the `fdtd` subdirectory separates general types and functions from declarations for the one-dimensional simulation in the `Ez` subdirectory.

The `src` directory closely follows the organization of the public headers because most declarations have a corresponding implementation file. In addition to the library code, it contains the `main.c` simulation program and the `benchmark.c` performance measurement program.

The `tools` directory contains scripts for repeating measurements, collecting system information, and plotting results. The `plot_sim.py` and `plot_benchmark.py` scripts process CSV files produced by the executables, while `plot_figures.py` creates illustrations that do not depend on execution results. The `docs` directory contains the English Markdown sources, metadata, and copies of the results used to build this document.

D.2 Distribution Directory

Building the project creates the `dist` directory containing the files needed to use the library. It contains no source or temporary object files and therefore forms a self-contained distribution intended for users. The `make docs` target additionally places the generated PDF in its `docs` subdirectory.

```
dist/
|-- bin/                           Executable directory
|   |-- tools/                     Utility scripts
|   |   |-- benchmark.sh           Benchmark repetition
|   |   |-- get_sys_info.sh        System-information collection
|   |   |-- plot_benchmark.py      Performance plots
|   |   |-- plot_sim.py            Simulation plots
|   |   `-- plot_figures.py        Standalone illustrations
|   |-- benchmark                   Performance measurement executable
|   `-- fdtd                        Simulation executable
|-- docs/
```

```
|   `-- Fdtd_Thesis.pdf            Generated English document
|-- include/                        Public headers
|   |-- fdtd/
|   |   |-- Ez/
|   |   |   |-- fdtd_ez_coeffs.h   Update-coefficient API
|   |   |   |-- fdtd_ez_fields.h   Field API
|   |   |   |-- fdtd_ez_kernels.h  Serial and OpenMP kernel API
|   |   |   |-- fdtd_ez_material.h Material API
|   |   |   |-- fdtd_ez_sim.h      Simulation API
|   |   |   |-- fdtd_ez_sources.h  Source API
|   |   |   `-- fdtd_ez_state.h    State API
|   |   |-- fdtd_accel.h           Acceleration modes
|   |   |-- fdtd_alloc.h           Memory allocation
|   |   |-- fdtd_const.h           Physical constants
|   |   |-- fdtd_ez.h              Ez module
|   |   `-- fdtd_types.h           Data types
|   `-- fdtd.h                      Public API entry point
|-- lib/
|   `-- libfdtd.a                   Static FDTD library
`-- buildinfo                       Compiler and build information
```

The `bin` directory contains the `fdtd` simulation executable and the `benchmark` performance measurement program. Its `tools` subdirectory contains the same utility scripts as the source project, allowing results to be produced and processed from the distribution.

The `include` directory contains a copy of the public headers and retains their original organization. A user program includes them at compile time to access the library's types, constants, and functions.

The `lib` directory contains the `libfdtd.a` static library, which is linked into a user program. The `buildinfo` file at the distribution root records the compiler and compilation options, allowing the build environment to be checked later. The `docs` directory contains the PDF produced from the Markdown sources.

D.3 Results Directory

Running the simulation, measurement, and image-generation targets creates the `results` directory. It combines raw CSV data, plots derived from those data, and standalone illustrations required to describe the model.

```
results/
|-- images/                         Generated plots
|   |-- benchmark_efficiency.png   Parallel efficiency
|   |-- benchmark_speedup.png      Speedup by thread count
|   |-- benchmark_speedup_k.png    Speedup by grid width
|   |-- benchmark_throughput.png   Throughput by thread count
|   |-- benchmark_throughput_k.png Throughput by grid width
|   |-- figure_fork_join.png       Thread fork and join
|   |-- figure_yee.png             Yee grid
|   |-- gauss_plot2d_Ez.png        Electric field Ez
```

```
|   |-- gauss_plot2d_Hy.png        Magnetic field Hy
|   |-- gauss_plot2d_ce1.png       Coefficient Ce1
|   |-- gauss_plot2d_ce2.png       Coefficient Ce2
|   |-- gauss_plot2d_ch1.png       Coefficient Ch1
|   |-- gauss_plot2d_ch2.png       Coefficient Ch2
|   |-- gauss_plot2d_eps_r.png     Relative permittivity
|   |-- gauss_plot2d_mu_r.png      Relative permeability
|   |-- gauss_plot2d_sigma_e.png   Electrical conductivity
|   |-- gauss_plot2d_sigma_m.png   Magnetic conductivity
|   |-- gauss_plot3d_EzHy.png      Spatial Ez and Hy field plot
|   `-- gauss_pml.png              PML positions
|-- benchmark.csv                  Performance measurements
`-- gauss.csv                      Gaussian-source simulation state
```

The `gauss.csv` file contains the state of a one-dimensional simulation with a Gaussian source, while `benchmark.csv` contains system information and individual timing measurements. The first file is used to verify physical behavior, and the second to calculate throughput, speedup, and efficiency.

The `images` directory contains plots of the fields, materials, coefficients, and PML, together with the performance plots. The `figure_*` images are produced separately by the `make figures` target and do not require a simulation or benchmark run.

D.4 Data Structures

The data structures separate the responsibilities of the individual parts of the one-dimensional simulation and connect them through a central object. Their declarations from the library's public headers are presented below, proceeding from the complete simulation to its constituent parts.

The `Fdtd_EzSim` structure represents the entire one-dimensional simulation and contains pointers to all of its components. It owns the state, fields, material, coefficients, and source list, so public simulation functions receive a single pointer to this object.

```
struct Fdtd_EzSim {
    Fdtd_EzState    *fsim_state;
    Fdtd_EzFields   *fsim_fields;
    Fdtd_EzMaterial *fsim_material;
    Fdtd_EzCoeffs   *fsim_coeffs;
    Fdtd_EzSources  *fsim_sources;
};
```

The `Fdtd_EzState` structure stores the global simulation parameters: the number of cells, spatial and temporal increments, Courant number, current time, and acceleration mode. Keeping these values in one place simplifies their updating and use by different kernels.

```
typedef struct {
    size_t fs_width;
    Fdtd_FloatType fs_dx;
```

```
    Fdtd_FloatType fs_dt;
    Fdtd_FloatType fs_courant;
    Fdtd_FloatType fs_time;
    Fdtd_AccelType fs_accel;
} Fdtd_EzState;
```

The `Fdtd_EzFields` structure contains the grid width and arrays of discrete magnetic-field H_y and electric-field E_z values. Both arrays contain one element for each cell, while their physical staggering determines how their indices are used in the update equations.

```
typedef struct {
    size_t ff_width;
    Fdtd_FloatType *ff_hy;
    Fdtd_FloatType *ff_ez;
} Fdtd_EzFields;
```

The `Fdtd_EzMaterial` structure describes the spatial distribution of material properties and losses. Arrays of permeability, permittivity, and the two conductivities allow every cell to have separate properties and the PML to be configured without changing the computational kernel.

```
typedef struct {
    size_t fm_width;
    Fdtd_FloatType *fm_mu;
    Fdtd_FloatType *fm_eps;
    Fdtd_FloatType *fm_sigma_m;
    Fdtd_FloatType *fm_sigma_e;
} Fdtd_EzMaterial;
```

The `Fdtd_EzCoeffs` structure stores four arrays of precalculated field-update coefficients. They are recalculated after a change in discretization, material, or boundary condition, removing division and material-property processing from the time loop.

```
typedef struct {
    size_t fc_width;
    Fdtd_FloatType *fc_ch1;
    Fdtd_FloatType *fc_ch2;
    Fdtd_FloatType *fc_ce1;
    Fdtd_FloatType *fc_ce2;
} Fdtd_EzCoeffs;
```

The `Fdtd_EzSources` structure manages a dynamic array of registered sources. It stores a pointer to its elements, the available capacity, and the number of occupied entries, allowing all sources to be applied in order after the electric field has been updated.

```
typedef struct {
    Fdtd_EzSource *fsrc_items;
    size_t fsrc_size;
    size_t fsrc_count;
} Fdtd_EzSources;
```

Appendix E

Using the Software

The built distribution in the `dist` directory allows the software to be used either as a library or as a set of ready-made programs. This appendix presents the basic procedures for running a simulation, plotting its results, and measuring performance.

E.1 Using the Library

The `dist/include` directory contains the public `fdtd.h` header and the other library headers, while the `dist/lib` directory contains the `libfdtd.a` static library. A user program includes `fdtd.h`, configures a simulation through the public functions, and finally releases the object with `Fdtd_EzSimDestroy()`. At compile time, the `-I` and `-L` options specify the paths to the headers and library.

The following example creates a grid of 200 cells, configures a PML, adds a Gaussian source, and performs 100 time steps. It writes the resulting state to `simulation.csv` for later analysis or resumption.

```
#include <math.h>
#include <fdtd.h>

static void Pulse(Fdtd_EzSim *sim, Fdtd_EzSourceData *data)
{
    Fdtd_FloatType n = Fdtd_EzSimGetTime(sim);
    Fdtd_FloatType pulse =
        exp(-0.5 * pow((data->f1 - n) / data->f2, 2.0));
    Fdtd_EzSimInjectEz(sim, (size_t) data->f3, pulse);
}

int main(void)
{
    // Create a simulation with K = 200 cells
    Fdtd_EzSim *sim = Fdtd_EzSimCreate(200);

    // Configure a PML twenty cells wide
    Fdtd_EzSimSetPML(sim, 20);

    // Create a Gaussian source
```

```
    Fdtd_EzSourceData source = {
        .f1 = 40.0,    // Time of the pulse maximum (n0)
        .f2 = 12.0,    // Pulse width (w)
        .f3 = 100.0    // Source position (kc = 100)
    };

    Fdtd_EzSimAddSource(sim, Pulse, source);
    Fdtd_EzSimRun(sim, 100);
    Fdtd_EzSimWriteCSV(sim, "simulation.csv");
    Fdtd_EzSimDestroy(sim);
    return 0;
}
```

The example is compiled by linking against `libfdtd` with `-lfdtd` and the mathematics library with `-lm`. The `-fopenmp` option provides the OpenMP support required by the built library, while `-O3` enables a high optimization level. After compilation succeeds, the resulting program is run from the same directory.

```
gcc -O3 -fopenmp -Idist/include example.c \
    -Ldist/lib -lfdtd -lm -o example
./example
```

E.2 Using the Program

The `dist/bin/fdtd` executable makes it possible to configure and run a simulation without writing a C program. Its options specify the grid width, discretization, materials, boundary condition, source, acceleration mode, and number of time steps. The complete list is available by running `dist/bin/fdtd --help`.

In the following example, we configure 200 cells, a PML twenty cells wide, and a Gaussian source, then perform 100 steps. If an output path is provided, the program writes the complete state to a CSV file; otherwise, it writes the state to standard output.

The `dist/bin/tools/plot_sim.py` script creates plots of the fields, materials, conductivities, and coefficients from a CSV state. The `-O` option selects the output directory, while `--prefix` and `--dpi` can change the filename prefix and plot resolution. Figure E.1 shows the `gauss_plot2d_Ez.png` file as an example of the script's output.

```
dist/bin/fdtd \
    --set-width=200 \
    --set-pml=20 \
    --add-pulse \
    --run=100 \
    simulation.csv
dist/bin/tools/plot_sim.py -O images simulation.csv
```

E.3 Using the Benchmark

The `dist/bin/benchmark` program measures the serial and OpenMP kernels for a grid width specified with `-K` and a number of time steps specified with `-N`. The `--max-threads`

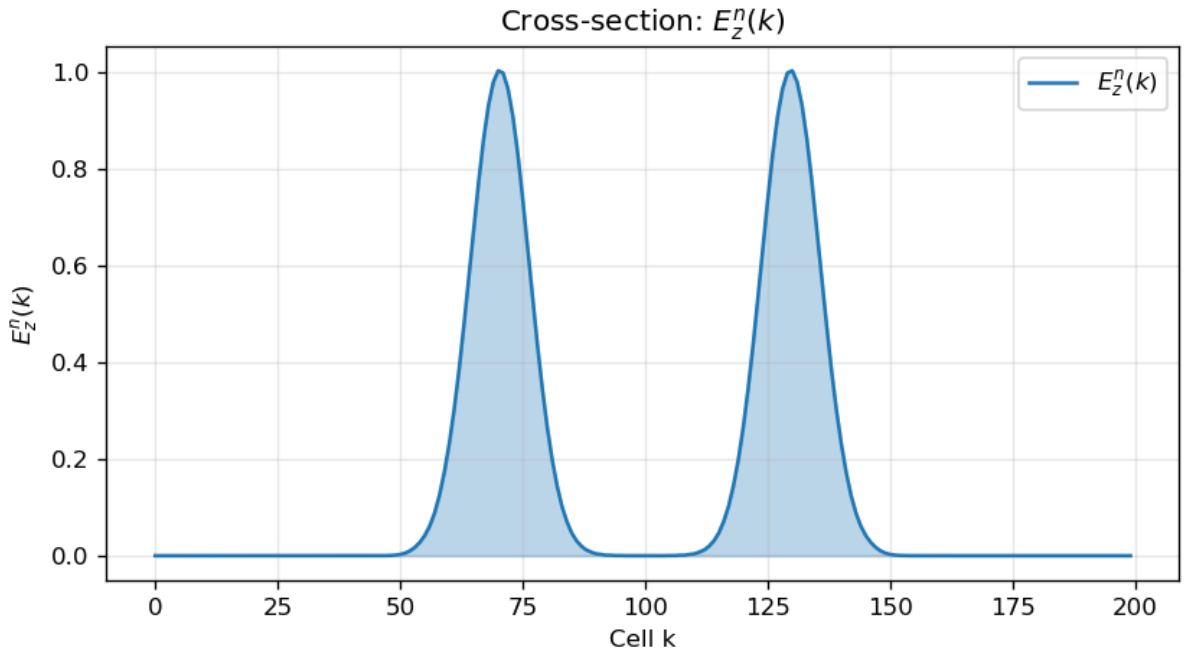


Figure E.1: Electric field E_z produced by the simulation plotting script

and `--repeat` options determine the largest thread count and the number of repetitions, while the result is written in CSV format.

The `dist/bin/tools/get_sys_info.sh` script retrieves information about the processor, cache memory, OpenMP environment, and other properties of the system. It writes this information as comments that can form the header of the `benchmark.csv` file. The benchmark output is appended to preserve those comments.

```
dist/bin/tools/get_sys_info.sh > benchmark.csv
dist/bin/benchmark \
    -K 80000 \
    -N 1024 \
    --max-threads=16 \
    --repeat=10 \
    >> benchmark.csv
```

The `dist/bin/tools/benchmark.sh` script repeats the same procedure for multiple grid widths and invokes the `benchmark` program supplied with the `-p` option. It writes results to standard output, which can be redirected to `benchmark.csv`. The `-n` option specifies the number of simulation steps, while `-r` specifies the number of measurement repetitions.

The `dist/bin/tools/plot_benchmark.py` script reads the resulting measurements and calculates throughput, speedup, and efficiency for every configuration. As with the simulation plots, the `-O` option selects the directory in which the images are written. Figure E.2 shows `benchmark_speedup.png` as an example of the script's output.

```
dist/bin/tools/get_sys_info.sh > benchmark.csv
dist/bin/tools/benchmark.sh \
    -p dist/bin/benchmark \
```

```
    -n 1024 \
    -r 10 \
    10000 20000 40000 80000 \
    >> benchmark.csv
dist/bin/tools/plot_benchmark.py -O images benchmark.csv
```

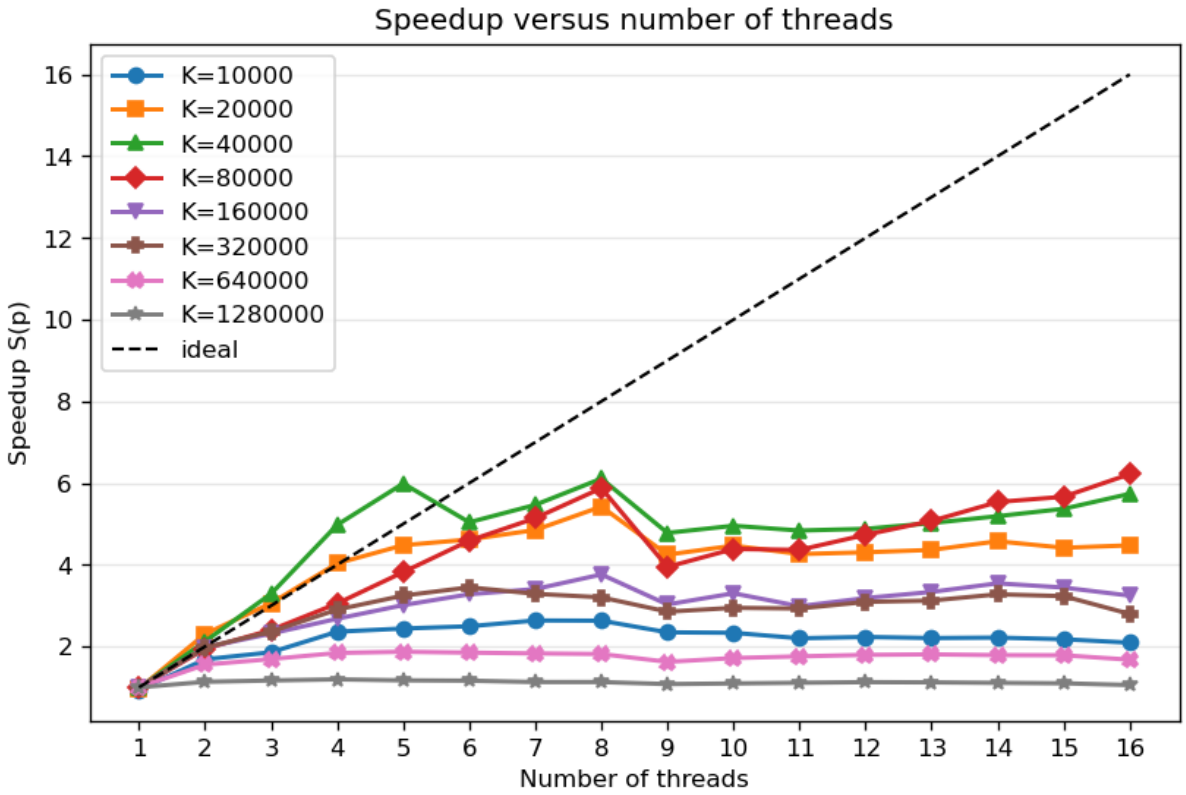


Figure E.2: Speedup of the OpenMP implementation produced by the benchmark plotting script